

Tribhuvan University

Faculty of Humanities and Social Sciences

Project Report

On

Student Progress Management System

Submitted to

Department of Computer Application

Xavier International College

Boudha, Kathmandu

In partial fulfillment of the requirements for the Bachelor in Computer Application (BCA)

Submitted by

Anish Baraili

TU Reg No: 6-2-546-21-2021

Pukar Parajuli

TU Reg No: 6-2-546-33-2021

BCA 8th Semester

Under the Supervision of

Mr. Ankit Nepal



Tribhuvan University

Faculty of Humanities and Social Sciences

Xavier International College

Boudha, Kathmandu

Supervisor's Recommendation

I hereby certify that the project entitled **"Student Progress Management System"** has been successfully completed by **Mr. Pukar Parajuli** and **Mr. Anish Baraili** under my direct supervision in partial fulfillment of the requirements for the Bachelor in Computer Application (BCA).

To the best of my knowledge, the work presented in this project report is original and has not been submitted, either wholly or partially, to any other institution for the award of any degree or academic qualification. The students have demonstrated dedication, technical competence, and the ability to apply theoretical concepts to solve real-world problems.
I am satisfied with the quality of the work and recommend this project report for evaluation and approval.

SIGNATURE

Mr. Ankit Nepal

SUPERVISOR

Humanities and Social Science

Bachelor in Computer Application

Xavier International College

Boudha, Kathmandu



Tribhuvan University Faculty of Humanities and Social Sciences

Xavier International College Boudha, Kathmandu

Letter of Approval

This is to certify that the project report entitled **"Student Progress Management System"**, submitted by **Mr. Pukar Parajuli** and **Mr. Anish Baraili**, has been examined and approved as a partial fulfillment of the requirements for the Bachelor in Computer Application (BCA).

The examination committee is satisfied that the project meets the academic standards and requirements prescribed by the institution.

----------	----------
Mr. Ankit Nepal	Mr. Roshan Jha
Supervisor	**Head of Department**
Bachelor in Computer Application	Bachelor in Computer Application
Xavier International College	Xavier International College
Boudha, Kathmandu	Boudha, Kathmandu
----------	----------
Internal Examiner	**External Examiner**

Acknowledgement

The successful completion of this project would not have been possible without the valuable support, guidance, encouragement, and cooperation of various individuals and institutions. We would like to express our sincere gratitude to everyone who contributed directly or indirectly to the successful completion of our project, “Student Progress Management System.”

We are highly indebted to Xavier International College for providing us with the necessary academic environment, resources, facilities, and support required for the successful development and completion of this project. The knowledge and skills gained throughout our academic journey played an important role in enabling us to undertake and complete this project successfully.

We would like to express our profound gratitude to our project supervisor, Mr. Ankit Nepal, for his continuous guidance, valuable suggestions, constructive feedback, and encouragement throughout the project. His supervision and support helped us understand the various aspects of project development and motivated us to improve the system and prepare this documentation in a systematic and standard manner.

We are also thankful to all the faculty members and teaching staff of Xavier International College for providing us with the necessary knowledge, guidance, and support throughout our academic journey. Their valuable suggestions and encouragement contributed significantly to the successful completion of this project.

We would also like to extend our sincere thanks to our friends, and colleagues for their valuable suggestions, technical support, feedback, and encouragement during the development of the project. Their assistance helped us overcome various challenges encountered during the implementation of the system.

Finally, we would like to express our deepest gratitude to our parents and family members for their continuous support, encouragement, patience, and motivation throughout the entire project period. Their support, along with the contribution of everyone mentioned above, has been invaluable in making this project a successful completion.

Project Submitted By

Pukar Parajuli

Anish Baraili

Abstract

The Student Progress Management System (SPMS) is a web-based application developed to manage and monitor students' academic progress in an organized way. The system provides role-based access for administrators, teachers, students, and parents to manage and view student information, academic grades, attendance records, and behavioral information through a centralized platform.

The system uses predefined rule-based algorithms and statistical calculations to analyze student performance and attendance. The grade average is calculated by dividing the total marks obtained by the number of assessments. Attendance percentage is calculated using the number of present days divided by the total recorded attendance. Based on these results, the system applies threshold-based conditions to classify academic performance, calculate a risk score, identify the risk level, and provide improvement suggestions. The risk score combines academic performance and attendance risk values to identify students who may require additional attention. The system also provides role-specific dashboards and generates student progress reports in PDF format for record keeping and review.

The system is developed using Python and Django, with SQLite for database management and ReportLab for PDF report generation. Overall, the project provides a structured way to maintain student records, analyze academic and attendance data, and support the monitoring of student progress. The analysis is based on predefined rules and statistical calculations rather than a trained machine learning model.

Keywords: *Student Progress Management System, Academic Performance, Attendance Management, Rule-Based Algorithm, Risk Score Calculation, Student Records, Django.*

Table of Contents

Supervisor's Recommendation ..... **i**
Letter of Approval ..... **ii**
Acknowledgement ..... **iii**
Abstract ..... **iv**
List of Abbreviations ..... **vii**
List of Figures ..... **viii**
List of Tables ..... **ix**
Chapter 1: Introduction **1**
1.1 Introduction 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope and Limitation 2
1.4.1 Scope 2
1.4.2 Limitations 3
1.5 Development Methodology 3
1.6 Report Organization 4
Chapter 2: Background Study and Literature Review **5**
2.1. Background Study 5
2.2 Literature Review 5
Chapter 3: System Analysis and Design **7**
3.1 System Analysis 7
3.1.1 Requirement Analysis 7
3.1.2 Feasibility Analysis 9
3.1.3 Object Modelling 10
3.1.4 Dynamic Modelling 12
3.1.5 Process Modelling 15
3.2 System Design 17
3.2.1 Refinement of System Models 17
3.2.2 Component Diagram 20
3.2.3 Deployment Diagram 21
3.3 Algorithm Details 22
3.3.1 Risk Score Calculation 22

3.3.2 Grade Performance Analysis....23
3.3.3 Attendance Pattern Analysis....24
3.3.4 Improvement Recommendation....24
3.3.5 Comprehensive Performance Analysis....25
Chapter 4: Implementation and Testing....26
4.1 Implementation....26
4.1.1 Tools and Technologies Used....26
4.1.2 Implementation Details of Modules....26
4.2 Testing....40
4.2.1 Unit Testing....40
4.2.2 System Testing....43
4.3 Result Analysis....47
4.3.1. Academic Performance Analysis....47
4.3.2. Attendance Analysis....48
4.3.3. Risk Analysis and Recommendations....48
4.3.4. Student Report Generation....48
4.3.5. Overall Result....49
Chapter 5: Conclusion and Future Recommendations....50
5.1 Lesson Learnt / Outcome....50
5.2 Conclusion....50
5.3 Future Recommendations....51
References....53
Appendices....54

List of Abbreviations

AI	Artificial Intelligence
CSS	Cascading Style Sheets
DB	Database
DFD	Data Flow Diagram
HTML	HyperText Markup Language
PDF	Portable Document Format
SPMS	Student Progress Management System
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator

List of Figures

Figure 1.1 Agile Methodology of Student Progress Management System 4
Figure 3.2 Use Case Diagram of Student Progress Management System 8
Figure 3.3 Gantt Chart of Student Progress Management System 10
Figure 3.4 Class Diagram of Student Progress Management System 11
Figure 3.5 Object Diagram of Student Progress Management System 12
Figure 3.6 State Diagram of Student Progress Management System 13
Figure 3.7 Sequence Diagram of Student Progress Management System 14
Figure 3.8 Activity Diagram of Student Progress Management System 15
Figure 3.9 System Flowchart of Student Progress Management System 16
Figure 3.10 Refinement of Class Diagram of Student Progress Management System 17
Figure 3.11 Refinement of Object Diagram of Student Progress Management System 18
Figure 3.12 Refinement of State Diagram of Student Progress Management System 18
Figure 3.13 Refinement of Sequence Diagram of Student Progress Management System 19
Figure 3.14 Refinement of Activity Diagram of Student Progress Management System 19
Figure 3.15 Component Diagram of Student Progress Management System 20
Figure 3.16 Deployment Diagram of Student Progress Management System 21

List of Tables

Table 3.1 Academic Risk Contribution of Student Progress Management System 22
Table 3.2 Attendance Risk Contribution of Student Progress Management System 22
Table 3.3 Grade Performance Classification of Student Progress Management System ..23
Table 3.4 Attendance Risk Classification of Student Progress Management System 24
Table 4.5 Tools and Technologies Used in Student Progress Management System 26
Table 4.6 Unit Testing of Student Progress Management System 41
Table 4.7 System Testing of Student Progress Management System 44
Table 4.8 Summary of Sample Data and Analysis Results of Student Progress Management System 49

Chapter 1: Introduction

1.1 Introduction

The use of computer technology in education has made it easier to store, manage, and access student-related information. Traditional methods of maintaining academic records using paper files or separate documents can make it difficult to organize information and monitor student progress over time. Important information such as marks, attendance, behavior, and teacher or parent comments may be stored separately, making regular monitoring more difficult.

The Student Progress Management System (SPMS) is a web-based system developed to manage and monitor the academic progress of students in a centralized environment. The system provides different access levels for administrators, teachers, students, and parents. Teachers can record student marks and attendance, while students and parents can view relevant academic information. Administrators can manage and monitor the overall information available in the system.

The system also provides performance analysis based on available academic and attendance data. It uses custom statistical and rule-based functions implemented in Python to calculate a risk score, identify performance levels, analyze attendance patterns, and provide improvement suggestions. These features help present student progress information in a more organized form.

The system also provides dashboards and student reports to make academic information easier to access. Student reports can be generated in PDF format for further use. Overall, the system is intended to provide a centralized platform for recording, monitoring, and analyzing student progress.

1.2 Problem Statement

Managing student academic progress manually can create difficulties in maintaining, accessing, and analyzing student records. Marks, attendance, behavior information, and other related records may be maintained separately, making it difficult to monitor students' overall progress.

The major problems addressed by the Student Progress Management System are:

- Difficulty in maintaining and accessing organized student academic records.
- Difficulty in monitoring student marks and attendance in one place.
- Difficulty in identifying students who may need additional academic attention.

- Difficulty in analyzing student performance and preparing organized progress reports.

The Student Progress Management System addresses these problems by providing a centralized web-based platform where authorized users can manage student information, record marks and attendance, analyze academic progress using predefined rules and statistical calculations, and generate student progress reports in PDF format.

1.3 Objectives

The main objective of the Student Progress Management System is to develop a web-based platform for managing and monitoring student academic progress.

The specific objectives are:

- To develop a web-based system for storing and managing student information, academic grades, and attendance records.
- To analyze students' academic performance and attendance using predefined rules and statistical calculations to identify students who may need additional attention.
- To provide role-based access, improvement suggestions, and student progress reports in PDF format.

1.4 Scope and Limitation

1.4.1 Scope

The scope of the Student Progress Management System (SPMS) is to provide a web-based platform for managing and monitoring student academic progress. The system focuses on the following areas:

- Providing role-based access for administrators, teachers, students, and parents.
- Managing student, class, and subject information.
- Recording and viewing student marks and attendance.
- Viewing available student behavior information.
- Analyzing academic performance and attendance using predefined rules and statistical calculations.
- Calculating student risk scores, classifying risk levels, and providing improvement suggestions.
- Displaying student progress through dashboards and generating reports in PDF format.

The system mainly focuses on student academic progress and related records. It does not cover all administrative activities of an educational institution.

1.4.2 Limitations

Although the Student Progress Management System provides features for managing and monitoring student progress, it has some limitations:

- The system does not include fee management, payroll, library management, or other general administrative functions.
- The accuracy and usefulness of the analysis depend on the marks and attendance records entered into the system.
- The analysis uses predefined rules and statistical calculations rather than a trained machine learning model.
- The system uses SQLite for database storage, which may require a more scalable database solution if the number of users and records increases significantly.
- The system may require further security improvements, testing, and configuration before being used in a real educational institution.
- The results are based on the available records and predefined conditions, so they may not fully represent every factor affecting a student's academic progress.

1.5 Development Methodology

Agile Model

The Agile Model is a software development approach in which the system is developed through repeated cycles of planning, development, testing, and improvement. It allows changes to be made during development based on identified requirements, testing results, and feedback.

The Agile Model was used for the development of the Student Progress Management System. The project was developed through repeated activities of requirement identification, system analysis, system design, implementation, testing, and improvement. The main functions were developed and tested progressively, and necessary changes were made during the development process. Project documentation was prepared and updated throughout the development process.

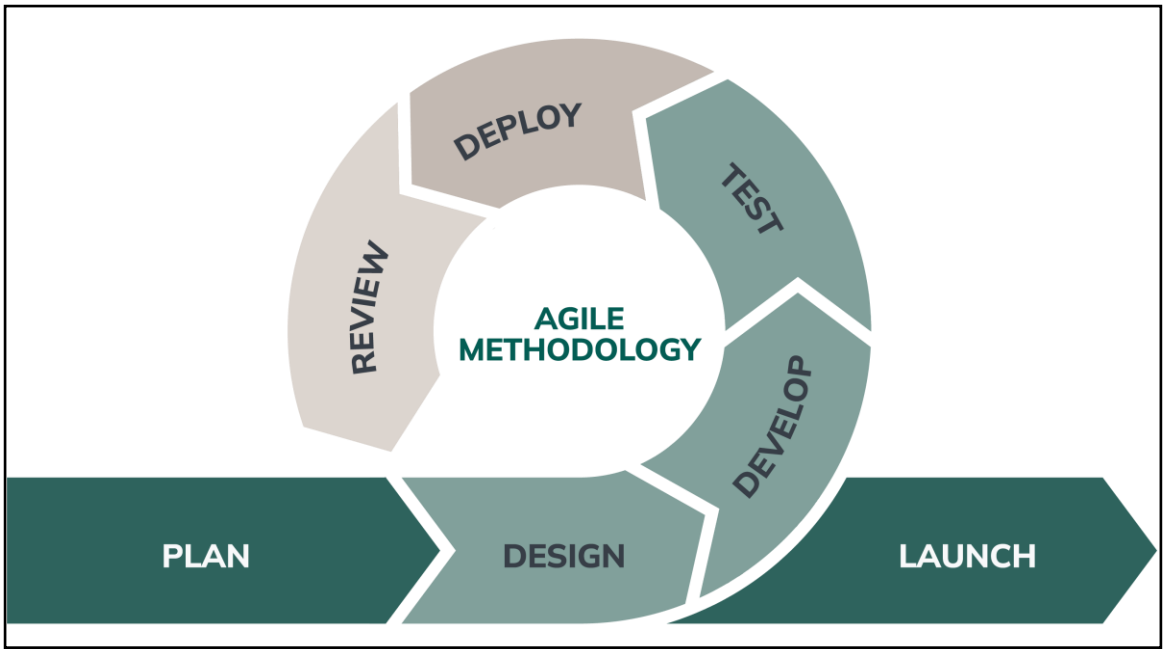


Figure 1.1 Agile Methodology of Student Progress Management System

1.6 Report Organization

The report is organized into five chapters.

Chapter 1: Introduction presents the introduction of the Student Progress Management System, problem statement, objectives, scope and limitations, development methodology, and organization of the report.

Chapter 2: Background Study and Literature Review presents the background concepts related to student progress management and reviews related studies and existing research relevant to the system.

Chapter 3: System Analysis and Design presents the system requirements, feasibility analysis, system analysis, object modelling, dynamic modelling, process modelling, system design, and algorithm details.

Chapter 4: Implementation and Testing presents the tools used, implementation details of the system modules, unit and system testing, and result analysis.

Chapter 5: Conclusion and Future Recommendations presents the learning outcomes of the project, the conclusion of the Student Progress Management System, and recommendations for further development of the system.

Chapter 2: Background Study and Literature Review

2.1. Background Study

The management of student academic information is an important part of educational activities. Traditional methods of maintaining student records through paper files or separate documents can make it difficult to organize, access, and monitor student progress. Information such as marks, attendance, behavior, and comments may be maintained separately, making it difficult to get a complete view of a student's academic progress.

The Student Progress Management System is a web-based system developed to manage and monitor student academic progress in a centralized environment. The system maintains information related to students, classes, subjects, marks, attendance, behavior, and comments. It provides different access based on the roles of administrators, teachers, students, and parents. Teachers can record marks and attendance, while students and parents can view relevant academic information and analysis. Administrators can manage and monitor the information available in the system.

The system also provides performance analysis using academic and attendance data stored in the database. Python functions use statistical calculations and predefined rules to calculate a student risk score, identify performance categories, analyze attendance patterns, and provide improvement suggestions. The analysis uses recent marks and attendance information and classifies students into levels such as High Risk, Medium Risk, Low Risk, and On Track. The system also provides dashboards for different users and generates student progress reports in PDF format. The analysis is based on predefined statistical and rule-based conditions and does not use a separately trained machine learning model. Therefore, the system provides a simple and organized approach to managing and analyzing student progress within the scope of an academic project.

2.2 Literature Review

Several studies have been conducted on educational information systems, attendance management, teacher-parent communication, and student performance analysis. These studies provide relevant background for the development of the Student Progress Management System.

Martins et al. studied Education Management Information Systems in higher education and highlighted the importance of information quality, system quality, user satisfaction, and

continued use [1]. This is relevant to the Student Progress Management System because it provides a centralized platform for managing student information.

Kraft and Rogers studied teacher-to-parent communication and its relationship with student outcomes [2]. This supports the importance of communication and sharing relevant student information between teachers, students, and parents.

Namoun and Alshanqiti reviewed data mining and learning analytics techniques for predicting student performance [3]. Similarly, the developed system uses student marks and attendance for performance and risk analysis, but applies predefined statistical and rule-based conditions instead of a trained machine learning model.

Bin Roslan and Chen reviewed educational data mining techniques for student performance prediction [4]. Their study supports the use of educational data for performance analysis, while the developed system uses a simpler rule-based approach suitable for the project scope.

Mohammed, Tolba, and Elmogy presented a Multimodal Student Attendance Management System using RFID and face recognition to improve attendance management [5]. This is relevant because attendance is also maintained as structured data in the developed system and used for performance analysis.

The reviewed studies provide relevant background for organizing student information, managing attendance, supporting teacher-parent communication, and analyzing student performance. The Student Progress Management System combines these areas in a single web-based platform and provides scope for future improvement through advanced data analysis and machine learning techniques.

Chapter 3: System Analysis and Design

3.1 System Analysis

For the Student Progress Management System, system analysis focuses on the requirements, user roles, data entities, system processes, and feasibility of the proposed application. The system is designed for four main user roles: administrator, teacher, student, and parent. The required functions and system behavior are analyzed according to the responsibilities of these users.

3.1.1 Requirement Analysis

Requirement analysis identifies the functional and non-functional requirements of the Student Progress Management System.

Functional Requirements

The functional requirements of the system are:

- **User Login and Role Management:** Users can log in and log out of the system, with access provided according to their roles as administrator, teacher, student, or parent.
- **Student Information Management:** The system maintains student details such as roll number, date of birth, class, parent information, contact information, and address.
- **Class and Subject Management:** The system maintains class and subject information required for managing student academic records.
- **Grade and Attendance Management:** Teachers can record student marks, maximum marks, examination type, and attendance.
- **Student Progress Information:** The system stores student behavior information and related records, which can be viewed where permitted.
- **Role-Based Dashboards:** Administrators, teachers, students, and parents can access dashboards according to their roles and available information.
- **Performance Analysis:** The system analyzes available student marks and attendance data to provide performance and attendance information.
- **Risk and Recommendations:** The system calculates a risk score, classifies the risk level using predefined conditions, and provides improvement suggestions.
- **Student Progress Viewing:** Authorized users can view student progress and related analysis based on their access.

- **PDF Report Generation:** The system generates student progress reports in PDF format containing relevant student records and analysis.

Use Case Diagram

The Use Case Diagram represents the interaction between the users and the functions of the Student Progress Management System.

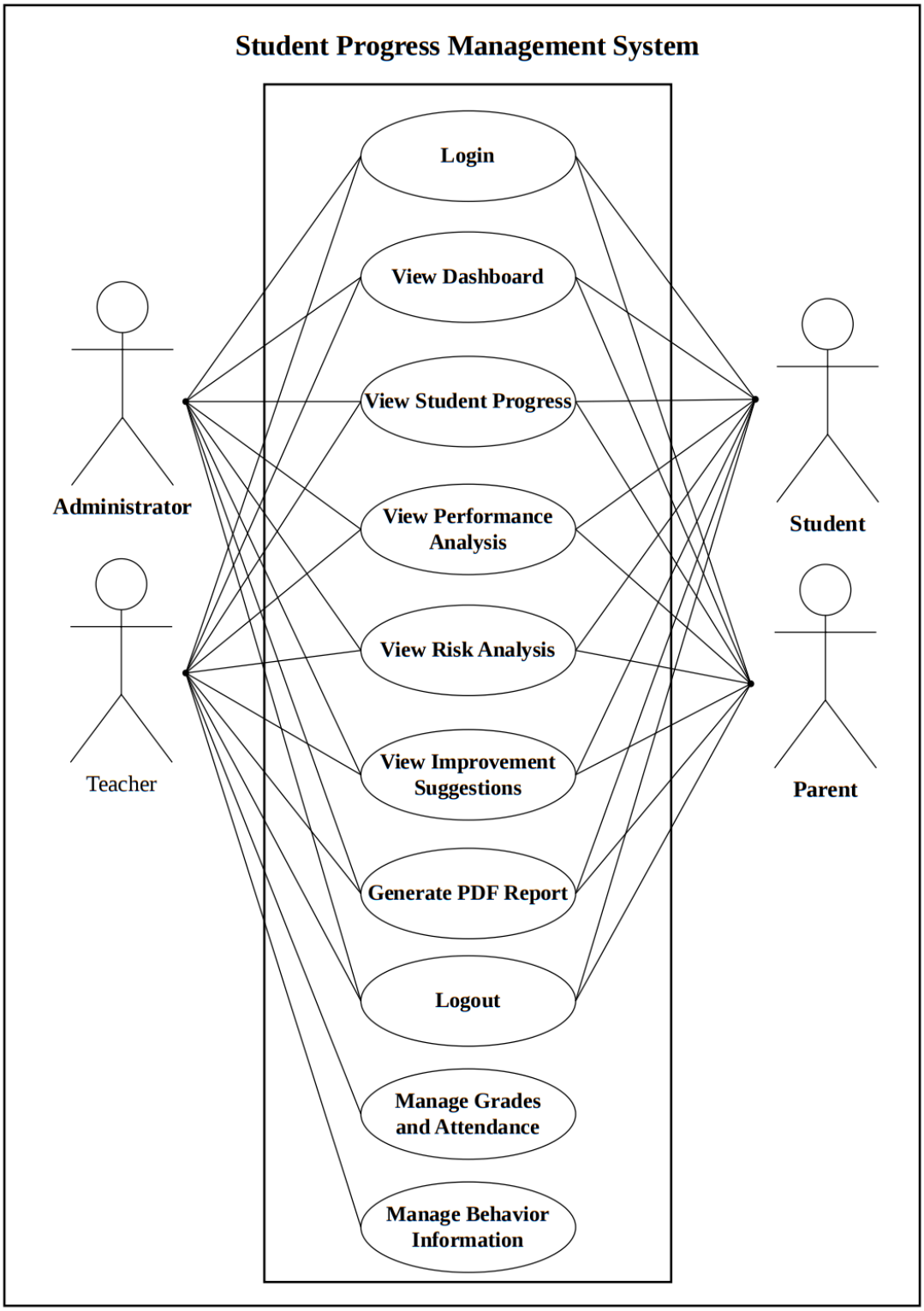


Figure 3.2 Use Case Diagram of Student Progress Management System

Non-Functional Requirements

The non-functional requirements of the system are:

- **Usability:** The system should provide a simple interface that can be used by administrators, teachers, students, and parents.
- **Performance:** The system should perform common operations such as login, record entry, dashboard access, and analysis within a reasonable time under normal use.
- **Reliability:** The system should correctly store and retrieve available student, academic, and attendance information during normal operation.
- **Security:** The system should restrict access to role-specific functions and information through authentication and role-based access control.
- **Maintainability:** The system structure should allow modules and analysis functions to be modified or extended when required.
- **Compatibility:** The web application should be accessible through a modern web browser.

3.1.2 Feasibility Analysis

Feasibility analysis determines whether the proposed system can be developed and operated using the available technical resources, operational environment, cost, and project schedule.

Technical Feasibility

The system is technically feasible because it uses Python and Django for web application development, HTML and CSS for the user interface, SQLite for data storage, and ReportLab for PDF report generation. The performance analysis functions are implemented using Python without requiring an external machine learning library. The technologies used are suitable for the development and execution of the academic project.

Operational Feasibility

The system is operationally feasible because its functions are related to common academic activities such as maintaining student information, recording marks and attendance, monitoring student progress, viewing analysis, and generating reports. Different users can access functions according to their assigned roles.

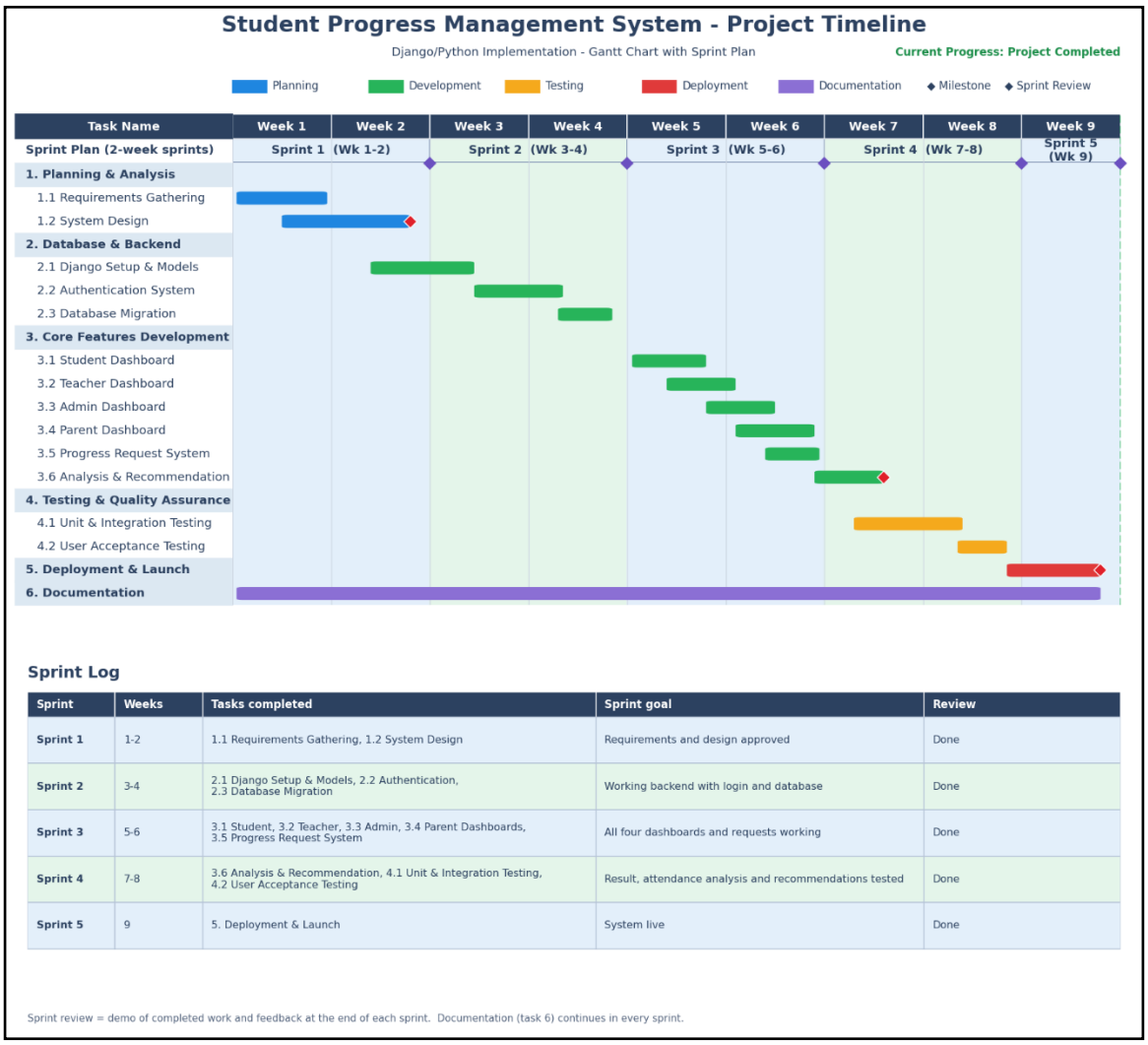
Economic Feasibility

The system is economically feasible because it uses commonly available hardware and open-source development technologies. Python, Django, SQLite, HTML, CSS, and

ReportLab do not require commercial software licenses for the development of the project. Therefore, the development cost is suitable for an academic project.

Schedule Feasibility

The system is schedule feasible because the development activities are organized into repeated cycles of requirement analysis, system design, implementation, testing, and improvement. The Agile Model supports an organized and flexible development process, allowing the system functions to be developed and improved progressively within the available project schedule.



Sprint Log

Sprint	Weeks	Tasks completed	Sprint goal	Review
Sprint 1	1-2	1.1 Requirements Gathering, 1.2 System Design	Requirements and design approved	Done
Sprint 2	3-4	2.1 Django Setup & Models, 2.2 Authentication, 2.3 Database Migration	Working backend with login and database	Done
Sprint 3	5-6	3.1 Student, 3.2 Teacher, 3.3 Admin, 3.4 Parent Dashboards, 3.5 Progress Request System	All four dashboards and requests working	Done
Sprint 4	7-8	3.6 Analysis & Recommendation, 4.1 Unit & Integration Testing, 4.2 User Acceptance Testing	Result, attendance analysis and recommendations tested	Done
Sprint 5	9	5. Deployment & Launch	System live	Done

Sprint review = demo of completed work and feedback at the end of each sprint. Documentation (task 6) continues in every sprint.

Figure 3.3 Gantt Chart of Student Progress Management System

3.1.3 Object Modelling

For the Student Progress Management System, object modelling represents the main entities involved in managing student information, academic records, attendance, behavior, and user roles, along with their relationships. The main objects represented in the system are User, Student, Teacher, Parent, Class, Subject, Grade, Attendance, Behavior, and Comment.

Class Diagram

The Class Diagram represents the main classes, their attributes, and relationships used in the Student Progress Management System.

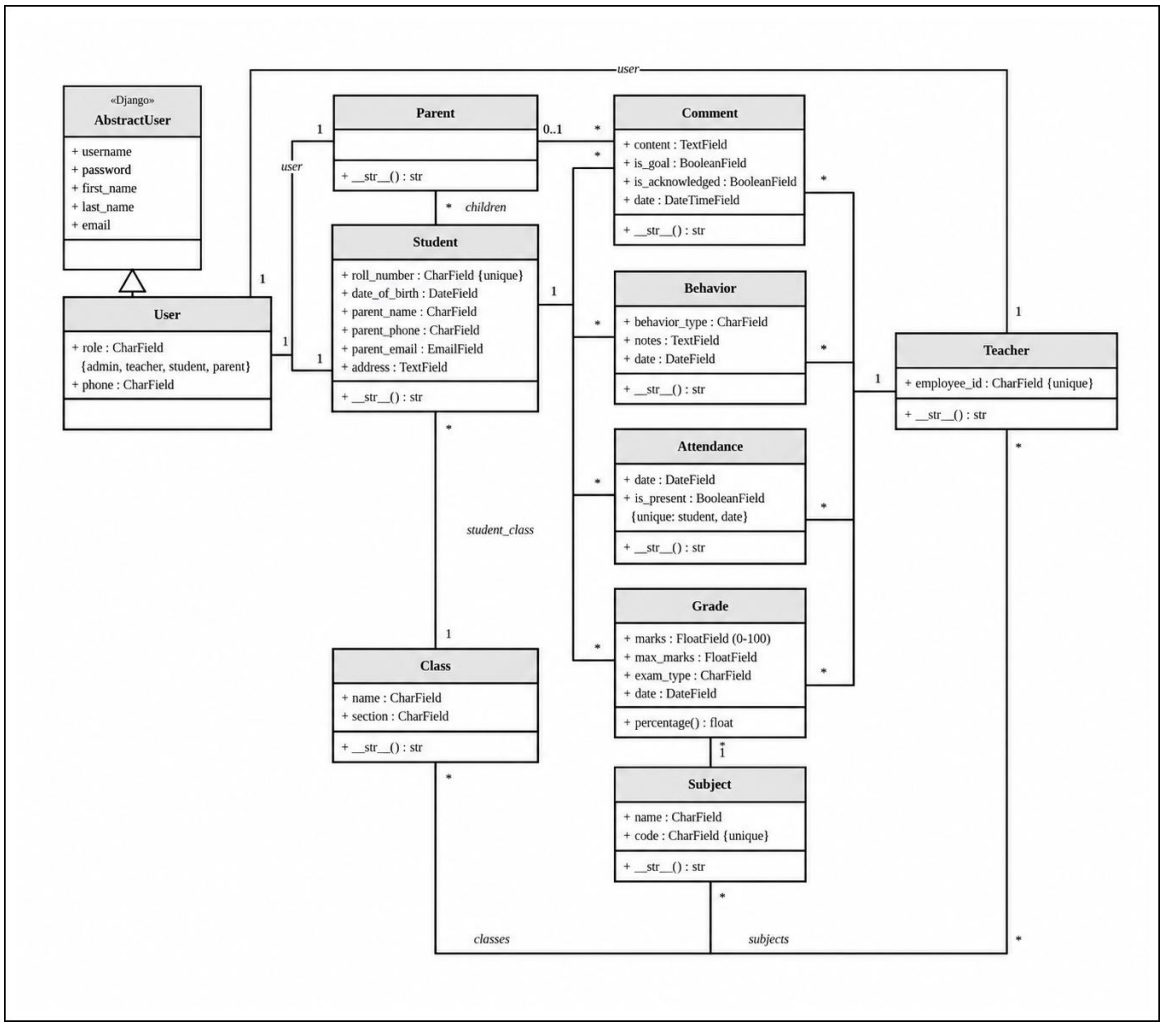


Figure 3.4 Class Diagram of Student Progress Management System

Object Diagram

The Object Diagram represents sample instances of the main classes and their relationships in the Student Progress Management System.

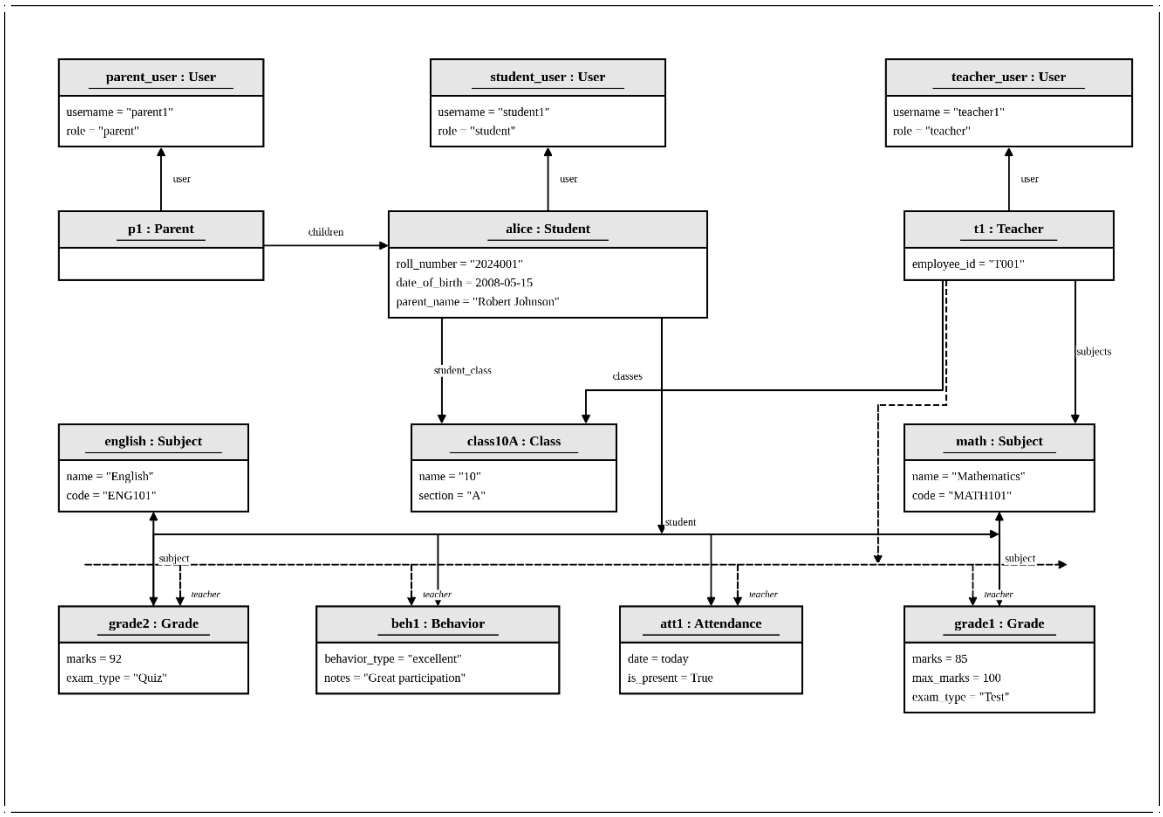


Figure 3.5 Object Diagram of Student Progress Management System

3.1.4 Dynamic Modelling

For the Student Progress Management System, dynamic modelling describes how users and system components interact during processes such as login, attendance recording, grade management, performance analysis, and report generation.

State Diagram

The State Diagram represents the different states and transitions involved in the operation of the Student Progress Management System, such as user interaction, data processing, and displaying results.

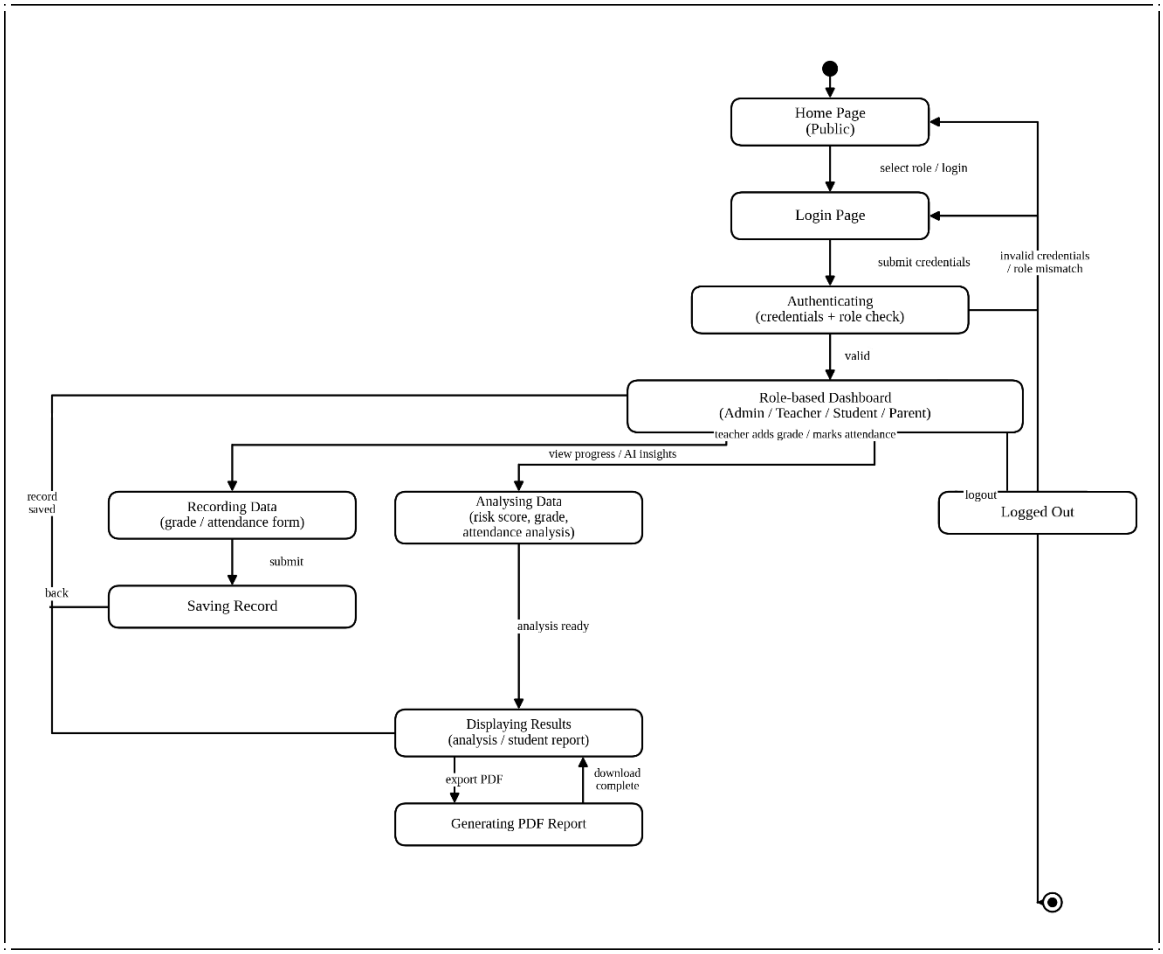


Figure 3.6 State Diagram of Student Progress Management System

Sequence Diagram

The Sequence Diagram represents the sequence of interactions between users, the system, database, analysis functions, and report generation during major system operations.

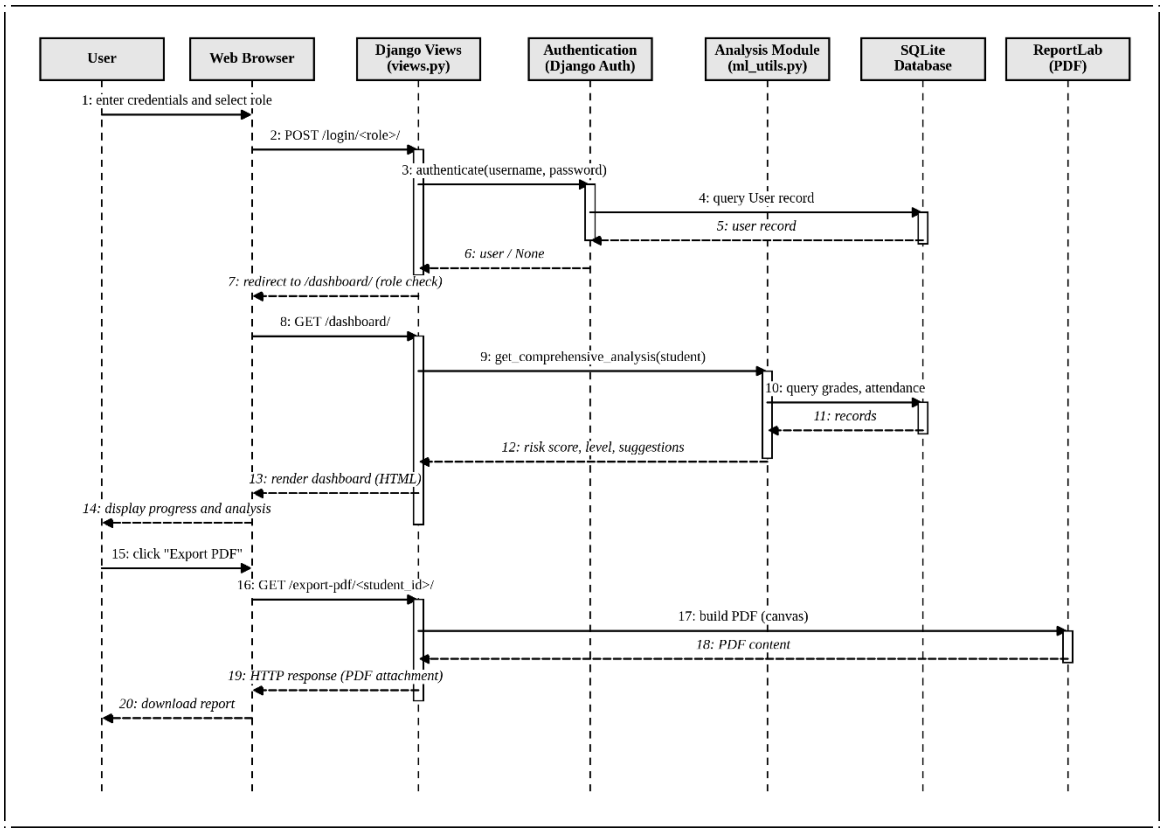


Figure 3.7 Sequence Diagram of Student Progress Management System

3.1.5 Process Modelling

Activity Diagram

In the Student Progress Management System, activity diagrams are used to represent the flow of major activities performed by the Administrator, Teacher, Student, and Parent.

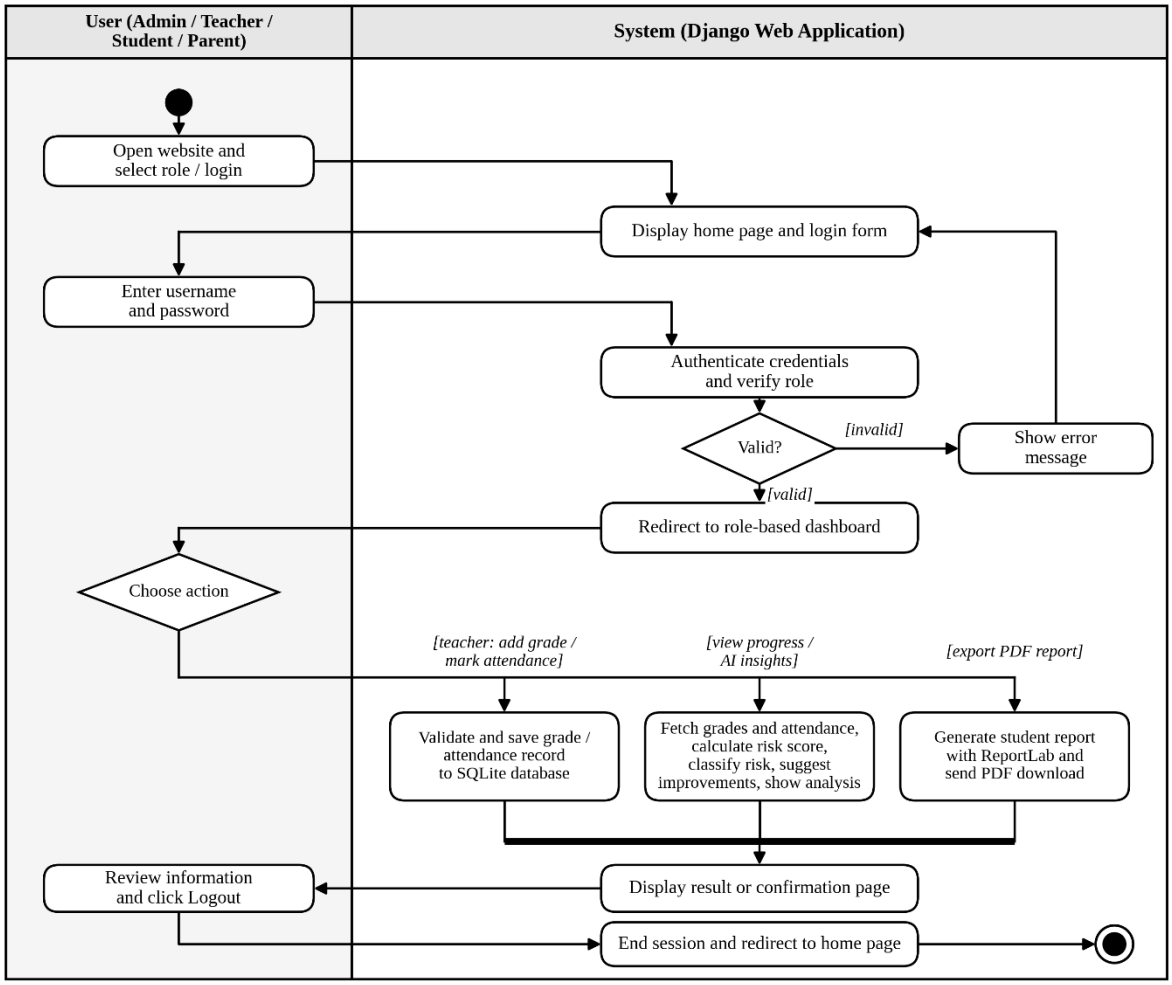


Figure 3.8 Activity Diagram of Student Progress Management System

System Flowchart

The System Flowchart shows the overall flow of the Student Progress Management System from user login to the completion of different system operations. The flow includes user authentication, role identification, access to the appropriate dashboard, management or viewing of student information, performance and attendance analysis, and PDF report generation.

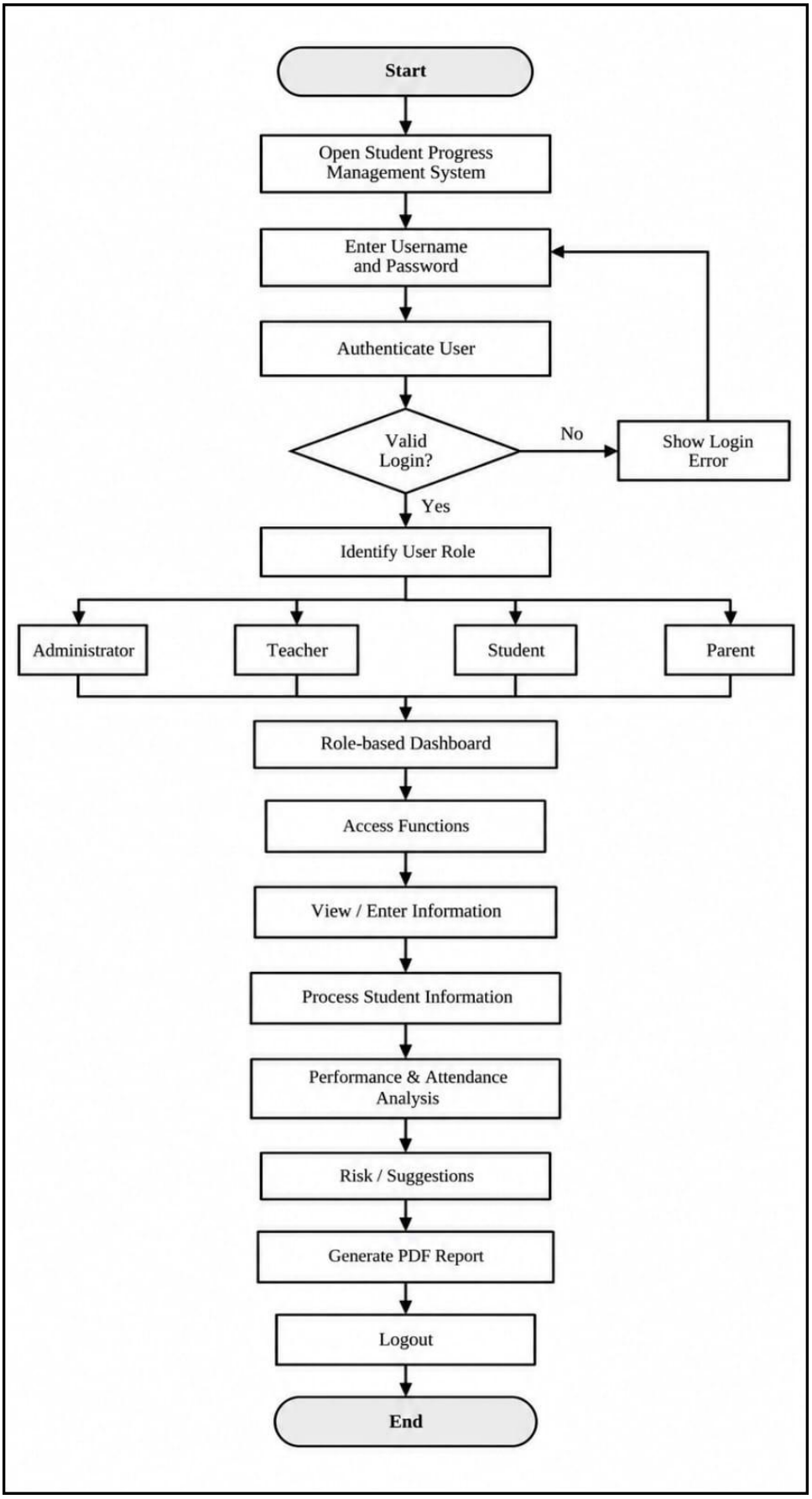


Figure 3.9 System Flowchart of Student Progress Management System

3.2 System Design

For the Student Progress Management System, system design defines the structure of the application, database, user roles, system modules, and interaction between the major components.

3.2.1 Refinement of System Models

The models developed during system analysis are refined during system design to represent the structure and behavior required for implementation. The Class, Object, State, Sequence, and Activity Diagrams are refined according to the system requirements and implemented functions.

The system uses Django models to represent the main data entities and their relationships. Student information is associated with classes, grades, attendance, behavior, and comments. Teachers are associated with subjects and classes, while parents are associated with their children.

The application processes user requests through Django views and uses the database models to store and retrieve information. The performance analysis functions process available academic and attendance information based on the implemented calculations and predefined rules. The reporting function generates student reports in PDF format.

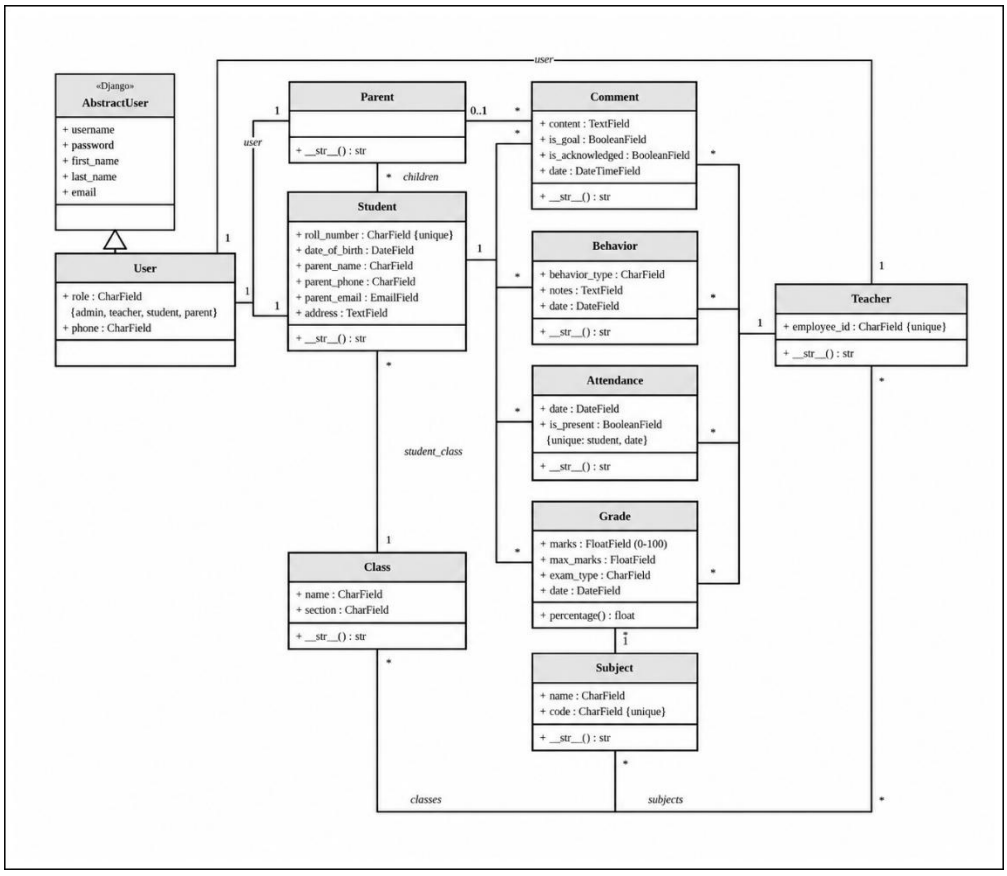


Figure 3.10 Refinement of Class Diagram of Student Progress Management System

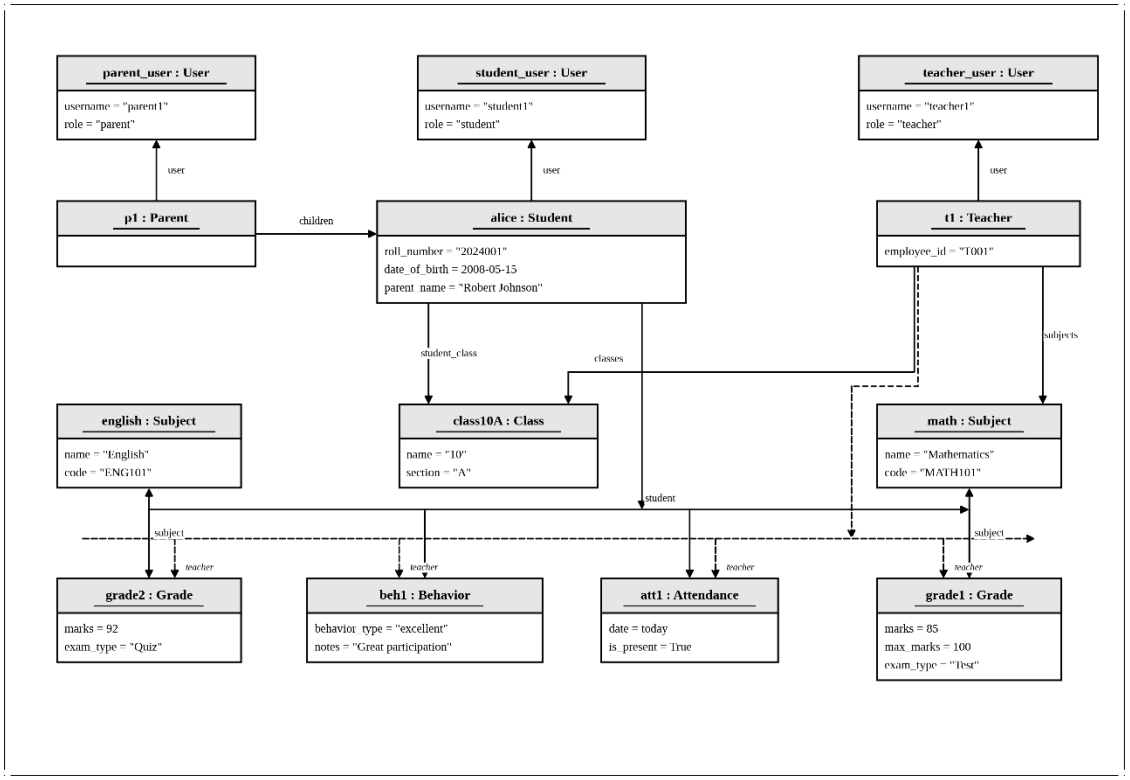


Figure 3.11 Refinement of Object Diagram of Student Progress Management System

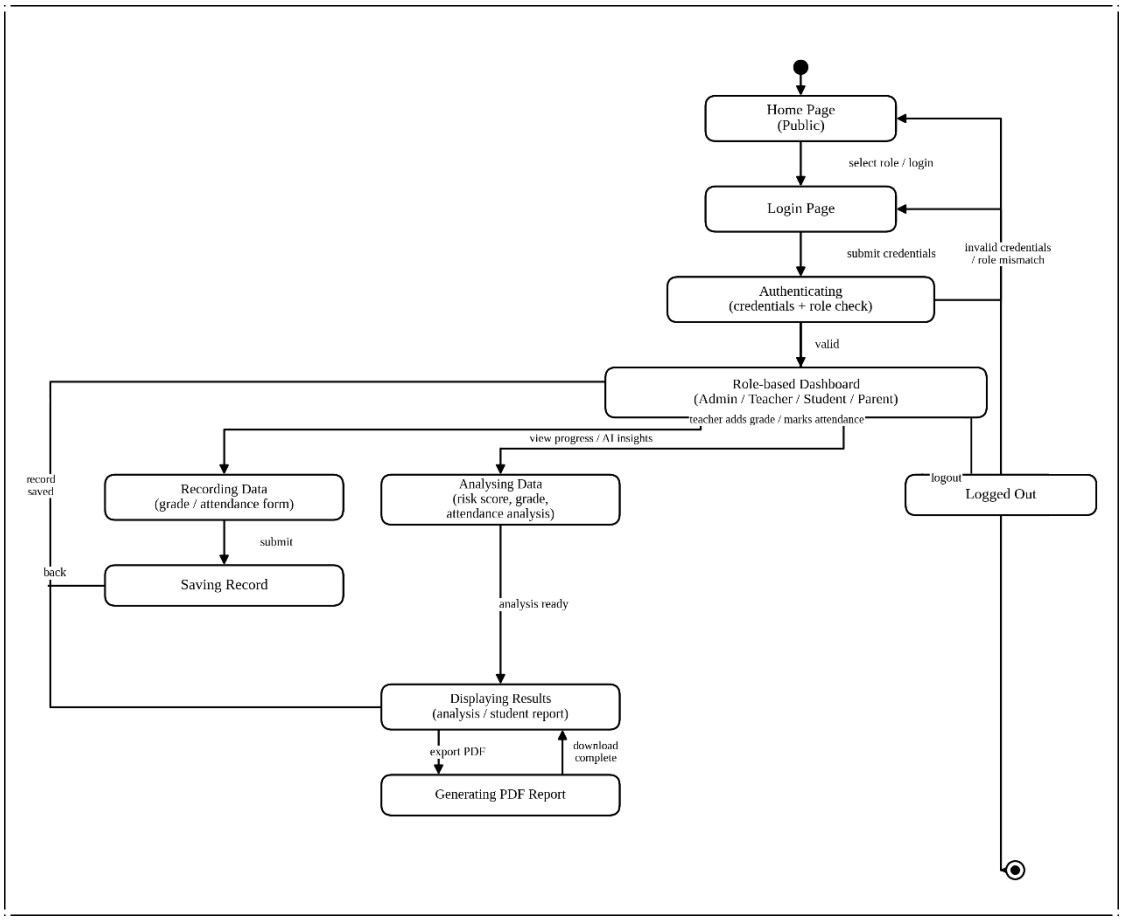


Figure 3.12 Refinement of State Diagram of Student Progress Management System

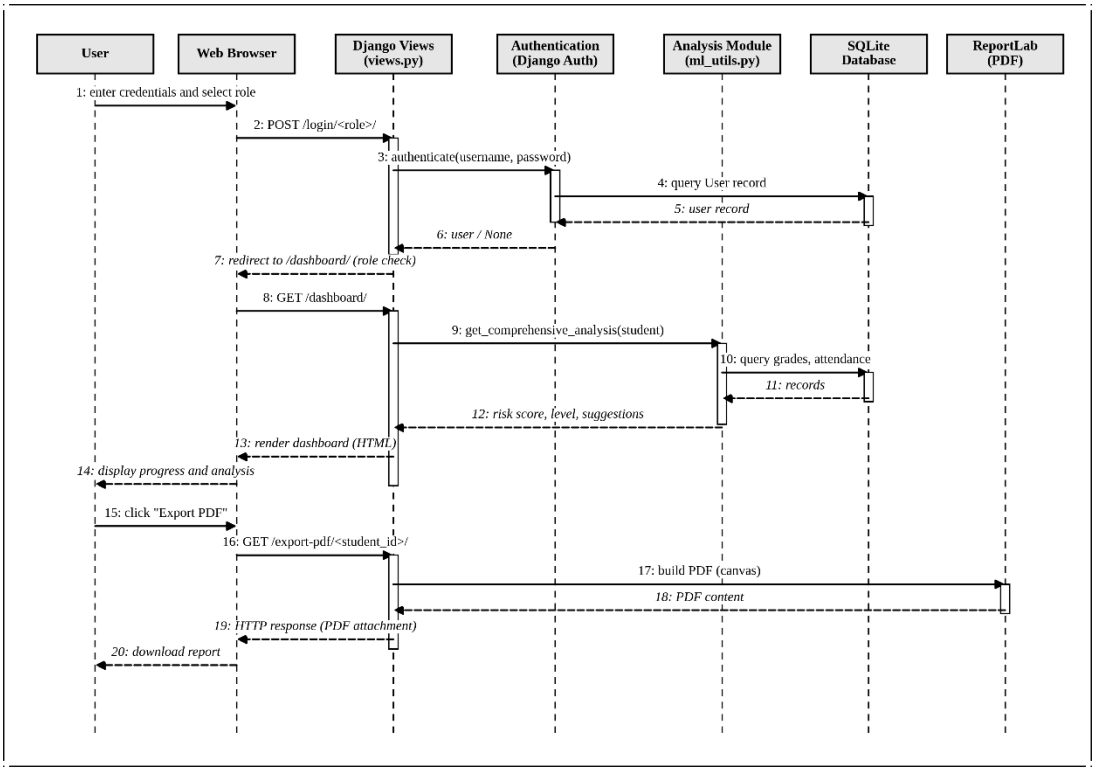


Figure 3.13 Refinement of Sequence Diagram of Student Progress Management System

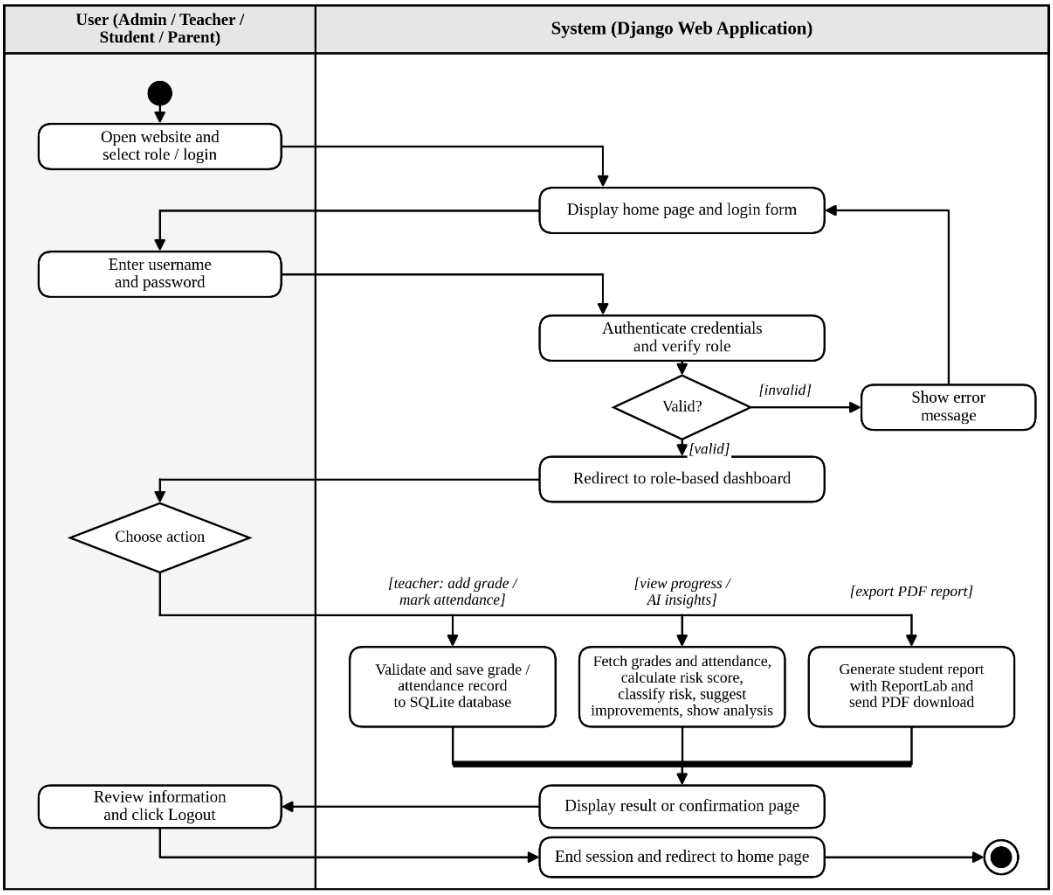


Figure 3.14 Refinement of Activity Diagram of Student Progress Management System

3.2.2 Component Diagram

The component diagram of the Student Progress Management System shows the major software components and their relationships, including the user interface, Django application components, database, authentication functions, analysis functions, and PDF report generation.

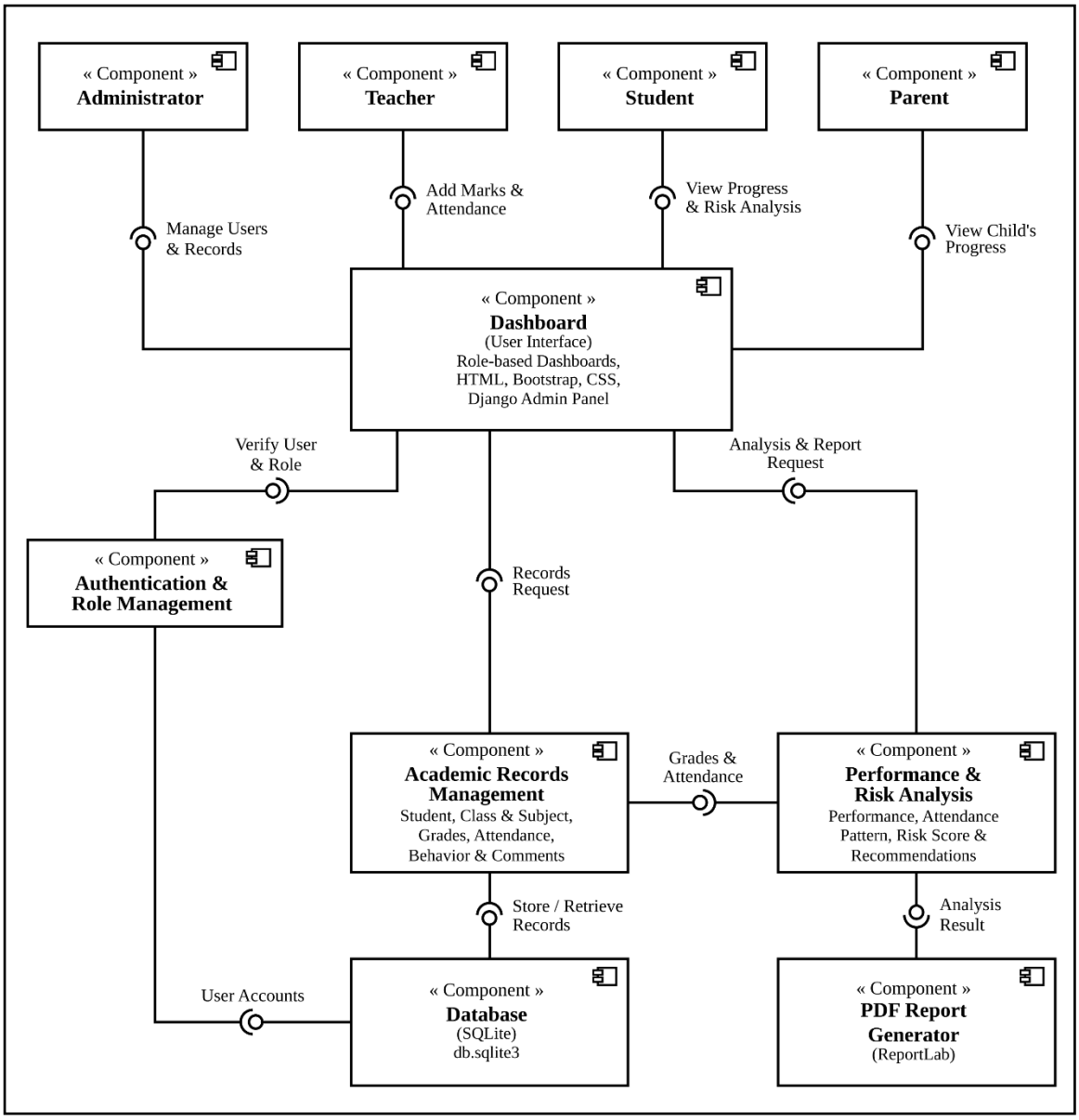


Figure 3.15 Component Diagram of Student Progress Management System

3.2.3 Deployment Diagram

For the Student Progress Management System, the deployment diagram represents how the web application is deployed and accessed through the user's device, web browser, application server, and database.

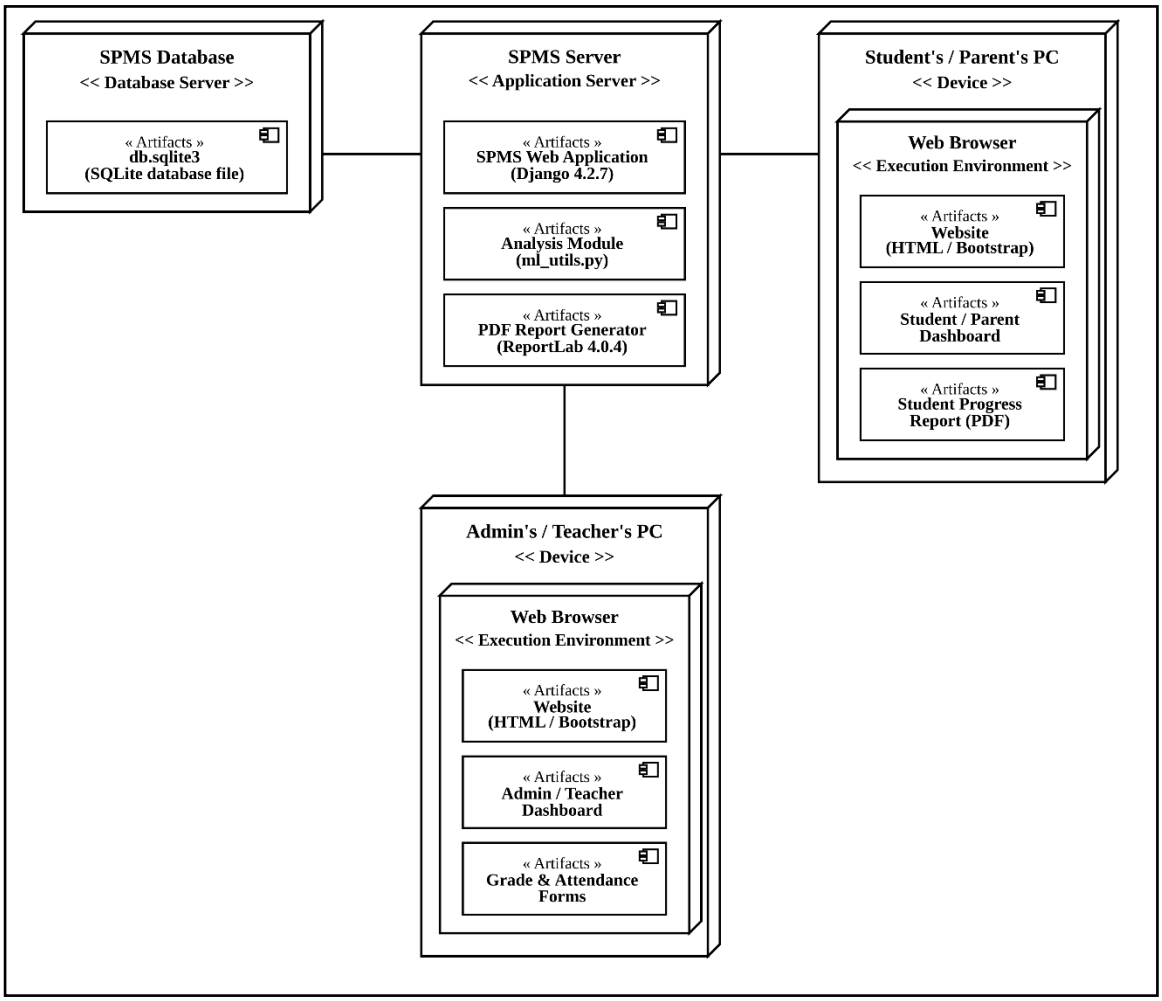


Figure 3.16 Deployment Diagram of Student Progress Management System

3.3 Algorithm Details

The Student Progress Management System uses statistical calculations and predefined rule-based conditions to analyze student performance, attendance, risk levels, and improvement suggestions. These functions are implemented in Python and use the available student grade and attendance records.

3.3.1 Risk Score Calculation

The system calculates a student risk score using recent grade and attendance information. Grade and attendance records from the recent 30-day period are considered.

The total risk score is calculated using the following formula:

[

R = \min(G + A, 100)

]

Where:

- (R) = Total risk score
- (G) = Academic risk points
- (A) = Attendance risk points.

Academic Risk Contribution

The academic risk points are determined from the student's average marks using predefined conditions:

Table 3.1 Academic Risk Contribution of Student Progress Management System

Average Marks	Risk Contribution
Less than 40	40
40 to less than 60	25
60 to less than 75	10
75 or above	0
No recent grades	20

Attendance Risk Contribution

The attendance risk points are determined from the student's attendance rate.

Table 3.2 Attendance Risk Contribution of Student Progress Management System

Average Rate	Risk Contribution
Less than 70%	35
70% to less than 85%	20
85% to less than 95%	8

95% or above	0
No recent attendance	15

Under the current predefined conditions, the maximum possible risk score is 75. Therefore, the upper limit of 100 in the formula does not affect the current calculated score.

The resulting score is classified using the following formula:

[
C(R)=
\begin{cases}
\text{High Risk}, & R\geq70\
\text{Medium Risk}, & 40\leq R<70\
\text{Low Risk}, & 20\leq R<40\
\text{On Track}, & R<20
\end{cases}
]

3.3.2 Grade Performance Analysis

The system calculates the average marks from the available grade records using the arithmetic mean formula:

[
M=\frac{\sum_{i=1}^{n}x_i}{n}
]

Where:

- (M) = Average marks
- (x_i) = Marks obtained in each assessment
- (n) = Number of assessments

The calculated average is classified according to the following conditions.

Table 3.3 Grade Performance Classification of Student Progress Management System

Average Marks	**Performance Level**
85 or above	Excellent
70 to less than 85	Good
50 to less than 70	Average
Less than 50	Below Average

An average below 70 is considered to require improvement. The analysis also provides the number of assessments available for the student.

3.3.3 Attendance Pattern Analysis

The system analyzes attendance records from the recent 60-day period. It calculates the number of present days, total recorded attendance days, and attendance rate.

The attendance rate is calculated using the following formula:

[
P=\frac{D_p}{D_t}\times100
]

Where:

- (P) = Attendance percentage
- (D_p) = Number of days present
- (D_t) = Total recorded attendance days

Attendance risk is classified as follows:

Table 3.4 Attendance Risk Classification of Student Progress Management System

Attendance Rate	Attendance Risk
95% or above	Low
85% to less than 95%	Medium
Less than 85%	High

When the attendance rate is below 80%, the system identifies a high risk of continued poor attendance. Otherwise, it indicates that good attendance is likely to continue.

3.3.4 Improvement Recommendation

The system generates improvement suggestions using predefined conditions based on average marks, attendance percentage, and risk score.

The recommendation process can be represented as:

[
Q=f(M,P,R)
]

Where:

- (Q) = Improvement recommendations
- (M) = Average marks
- (P) = Attendance percentage
- (R) = Total risk score

When the average grade is below 60, the system provides suggestions related to academic improvement and additional support. When attendance is below 85%, suggestions related to improving attendance are provided.

When the risk score is 70 or above, the system provides an immediate intervention suggestion. When the risk score is 40 or above, the system recommends closer monitoring. Otherwise, a positive progress-related suggestion is provided.

3.3.5 Comprehensive Performance Analysis

The system combines risk analysis, grade analysis, attendance analysis, and improvement suggestions to produce a comprehensive student analysis.

The overall status is determined according to the risk score:

[
S(R)=
\begin{cases}
\text{Needs Immediate Attention}, & R\geq70\\
\text{Monitor Closely}, & 40\leq R<70\\
\text{Minor Concerns}, & 20\leq R<40\\
\text{Excellent Progress}, & R<20
\end{cases}
]

Where (S(R)) represents the overall student progress status.

The resulting analysis is displayed to authorized users through the relevant system pages and can be included in the generated student PDF report.

Chapter 4: Implementation and Testing

4.1 Implementation

The Student Progress Management System is implemented as a web-based application using Python and the Django framework. The system provides different access and functions for administrators, teachers, students, and parents according to their roles. The implementation includes user authentication, student information management, academic record management, attendance management, performance analysis, and PDF report generation.

4.1.1 Tools and Technologies Used

The main tools and technologies used for the implementation of the Student Progress Management System are presented in Table:

Table 4.5 Tools and Technologies Used in Student Progress Management System

Tool/Technology	Purpose
Python	Main programming language used for backend logic and analysis functions
Django 4.2.7	Web application framework used for application development
HTML	Used to structure the web pages
CSS	Used to design and format the user interface
Bootstrap	Used for interface layout and responsive page elements
SQLite	Database used to store system information
ReportLab 4.0.4	Used to generate student progress reports in PDF format
Visual Studio Code	Development environment used for writing and managing the project code
Web Browser	Used to access and operate the web application

The project uses Django's built-in authentication and session facilities together with custom application models and views. The performance analysis functions are implemented in Python within the project and do not require an external machine learning library.

4.1.2 Implementation Details of Modules

The Student Progress Management System (SPMS) is implemented using Django and Python for the backend, HTML, CSS, and Bootstrap for the frontend, and SQLite for database management. Django's Model-View-Template (MVT) architecture is used to organize the application into models, views, and templates. The system is divided into

different modules to manage student information, attendance, grades, behavior, performance analysis, dashboards, and report generation.

Each module performs a specific task and works with other modules to maintain student records and display academic progress. Django's built-in authentication system, ORM, URL routing, and template system are used to handle user access, database operations, page navigation, and data presentation.

1. User Authentication and Role Management Module

The User Authentication and Role Management Module manages user login and access to different parts of the system. System allows authorized users to access the system according to their assigned roles, including Administrator, Teacher, Student, and Parent.

Django's authentication system is used to verify login credentials and manage authenticated users. After successful login, the system provides access to the appropriate dashboard and features based on the user's role. Unauthorized users should not be allowed to access restricted pages.

Classes, Functions, and Methods:

- **User Model:** Stores user account information and supports authentication.
- **Login and Logout Views:** Handle user login and logout requests.
- **Authentication Methods:** Check login status and validate access to protected pages.
- **Role-Based Access Logic:** Determines which pages and operations are available to each role.
- **URL Routing:** Connects authentication requests to the appropriate views.

Pseudocode: User Authentication

```
BEGIN
  Receive username and password
  Verify the login credentials
  IF credentials are valid THEN
    Create authenticated user session
    Identify the user's role
    Redirect user to the appropriate dashboard
  ELSE
    Display an error message
  END IF
END
```

2. Student Information Management Module

The Student Information Management Module maintains student records and provides the information required by other modules. It helps authorized users access student details and associate academic records with the correct student.

Student information is stored in the database and retrieved through Django's ORM. The ORM allows the application to perform database operations using Python objects instead of writing SQL queries for every operation.

The module supports the student information operations implemented in the system, such as displaying student details and retrieving records for academic progress reports.

Classes, Functions, and Methods:

- **Student-Related Model:** Represents student information stored in the database.
- **Django ORM Methods:** Retrieve and update student records.
- **View Functions:** Process requests for student details and prepare the information for display.
- **Template Files:** Display student information on the relevant pages.

Pseudocode: Retrieve Student Information

```
BEGIN
  Receive the student identification
  Check whether the user has permission

  IF access is permitted THEN
    Retrieve the student record from the database

    IF the record exists THEN
      Display student information
    ELSE
      Display a record-not-found message
    END IF
  ELSE
    Deny access
  END IF
END
```

3. Class and Subject Management Module

The Class and Subject Management Module organizes academic information used to maintain student records and evaluate academic performance. Class and subject information provide the necessary context for recording grades and preparing progress reports.

Django models and database relationships are used to connect academic information with the relevant student records. The ORM retrieves the required records when a teacher or another authorized user accesses an academic page.

This module helps maintain consistency when academic records are associated with students, classes, and subjects. The available operations depend on the functions implemented in the system.

Classes, Functions, and Methods:

- **Academic Models:** Represent the class, subject, or related academic information defined in the database.
- **ORM Query Methods:** Retrieve the relevant class and subject records.
- **View Functions:** Process requests for academic information.
- **Templates:** Display class and subject information.

Pseudocode: Retrieve Academic Information

```
BEGIN
  Receive the request for academic information
  Check user permissions

  IF access is permitted THEN
    Retrieve the required class and subject records
    Associate records with the relevant students
    Display the academic information
  ELSE
    Deny access
  END IF
END
```

4. Grade and Marks Management Module

The Grade and Marks Management Module manages students' academic results. Teachers can enter or update grades through the available grade management functions. The

information is stored in the database and used to display academic results and evaluate student performance.

The Django view responsible for grade management processes the submitted information, while the ORM performs the relevant database operations. Input validation and permission checks are important to ensure that grades are associated with the correct students and that only authorized users can modify them.

The stored grade information is also used by the student report and performance analysis features.

Classes, Functions, and Methods:

- **Grade-Related Model:** Represents the academic marks or grades stored in the database.
- **add_grade View or Route:** The grade-entry functionality is connected through the add-grade route in core/urls.py.
- **Django ORM Methods:** Store and retrieve grade records.
- **Validation Logic:** Checks the submitted information before saving it.
- **Report and Analysis Functions:** Retrieve grade information for academic reports and performance evaluation.

Pseudocode: Add or Update Student Grade

```
BEGIN
  Receive student, subject, and grade information
  Verify user permission
  Validate the submitted information

  IF the information is valid THEN
    Save or update the grade record
    Display a success message
  ELSE
    Display a validation error
  END IF
END
```

5. Attendance Management Module

The Attendance Management Module maintains student attendance records. Teachers can record attendance through the available attendance functionality, and the saved records can be used to review attendance status and identify attendance patterns.

The module uses Django views to process attendance requests and the ORM to save or retrieve attendance information. Attendance records are associated with the relevant students so that the system can retrieve them when preparing reports and performance analysis.

The attendance information also supports the rule-based analysis used to identify students who may need additional attention because of their attendance records.

Classes, Functions, and Methods:

- **Attendance-Related Model:** Represents attendance records in the database.
- **mark_attendance View or Route:** The attendance functionality is connected through the mark-attendance route in core/urls.py.
- **ORM Methods:** Save, update, and retrieve attendance records.
- **Attendance Analysis Logic:** Uses available attendance information to identify patterns.
- **Report Functions:** Retrieve attendance records for student reports.

Pseudocode: Record Student Attendance

```
BEGIN
  Receive student attendance information
  Verify user permission
  Validate the submitted records

  FOR each student attendance record
    Check the required information

    IF the record is valid THEN
      Save the attendance record
    ELSE
      Report the validation error
    END IF
  END FOR

  Display the attendance result
END
```

6. Student Behavior and Teacher Comments Module

The Student Behavior and Teacher Comments Module records information about students' behavior and development. It helps teachers maintain additional observations alongside academic results and attendance records.

The Behavior model stores behavior-related information. The behavior categories used in the project include Excellent, Good, Average, and Needs Improvement, with notes providing additional details where applicable.

The Comment model is used to store teacher comments separately from behavior records. Keeping these records separate allows the system to distinguish structured behavior information from written observations.

These records can support the review of student progress. However, the actual availability of comment entry and display depends on the views and templates implemented in the project.

Classes, Functions, and Methods:

- **Behavior Model:** Stores student behavior categories and related notes.
- **Comment Model:** Stores teacher comments as separate database records.
- **Django ORM Methods:** Retrieve and save behavior or comment records.
- **Admin Registration:** Makes the registered models available for management through Django's administration interface.
- **View and Template Logic:** Determines how behavior information and comments are entered and displayed.

Pseudocode: Retrieve Student Behavior Information

```
BEGIN
  Receive the student identification
  Verify access permission

  IF access is permitted THEN
    Retrieve the student's behavior records

    IF records are available THEN
      Display behavior categories and notes
    ELSE
      Display a message indicating no records
    END IF
```

ELSE

Deny access

END IF

END

Teacher comments should be described as a separate student progress feature. The report should not claim that comments are available on the parent dashboard unless the corresponding display functionality has been implemented.

7. Student Performance Analysis Module

The Student Performance Analysis Module evaluates available student information to provide a clearer view of academic progress. It uses stored grades, attendance records, and other relevant information to support the review of student performance.

The project uses predefined statistical calculations and rule-based conditions rather than a model trained on a dataset. The analysis follows the conditions defined in the application's Python logic and presents the resulting information through the relevant views and templates.

This module helps organize student information into meaningful results that can be reviewed by authorized users. The quality of the results depends on the completeness of the stored records and the conditions defined in the system.

Classes, Functions, and Methods:

- **Grade Retrieval Methods:** Obtain academic results from the database.
- **Attendance Retrieval Methods:** Obtain attendance information for analysis.
- **Analysis Functions:** Apply predefined calculations or conditions to the retrieved information.
- **View Functions:** Prepare the analysis results for display.
- **Templates:** Present the results to authorized users.

Pseudocode: Student Performance Analysis

BEGIN

Receive the student identification

Retrieve the student's grade records

Retrieve the student's attendance records

IF sufficient information is available THEN

Calculate the required academic measures

Apply the predefined performance conditions

Prepare the performance results

Display the analysis

ELSE

Display a message indicating insufficient information

END IF

END

The exact calculations and conditions depend on the analysis logic implemented in the project. The system should not be described as using a trained machine learning model unless such a model is actually implemented.

8. Student Risk and Recommendation Module

The Student Risk and Recommendation Module uses predefined rules to identify student records that may require further attention. It considers the information available in the system, such as academic performance and attendance, according to the conditions defined in the application's analysis logic.

The module does not independently diagnose a student's situation. Instead, it provides rule-based indications that can help teachers and other authorized users review student progress and decide whether additional support may be needed.

Recommendations or insights are generated according to the implemented conditions. Their accuracy and usefulness depend on the quality of the data and the suitability of those conditions.

Classes, Functions, and Methods:

- **Analysis Functions:** Evaluate available student information.
- **Rule-Based Conditions:** Determine whether specified conditions have been met.
- **Insight or Recommendation Logic:** Prepares the relevant results based on the defined rules.
- **View Functions:** Make the analysis available to the appropriate pages.
- **Templates:** Display the results to authorized users.

The core/urls.py file includes routes for ai-insights and student-ai-analysis. These route names are retained from the implementation, but the report describes the underlying analysis as rule-based rather than claiming that it uses trained artificial intelligence.

Pseudocode: Rule-Based Student Analysis

BEGIN

Retrieve the required student information

Check whether sufficient data is available

```
    IF sufficient data is available THEN
        Evaluate the predefined conditions

        IF a condition for additional attention is met THEN
            Prepare the relevant insight
        ELSE
            Prepare the normal progress result
        END IF

        Display the result
    ELSE
        Display a message indicating insufficient information
    END IF
END
```

9. Attendance Pattern Analysis Module

The Attendance Pattern Analysis Module reviews student attendance records to identify patterns that may be useful when evaluating progress. It uses the attendance information stored in the database and applies the calculations or conditions defined in the application. The analysis can help highlight attendance-related concerns when the available records meet the specified conditions. The resulting information can be used alongside grades and other student records to provide a broader view of student progress.

Classes, Functions, and Methods:

- **Attendance Model and ORM Methods:** Provide access to stored attendance records.
- **Attendance Analysis Functions:** Process the retrieved information.
- **Rule-Based Conditions:** Identify patterns according to the application's predefined logic.
- **View and Template Functions:** Present the results to authorized users.

Pseudocode: Attendance Pattern Analysis

```
BEGIN
    Retrieve attendance records for a student
    Check whether sufficient attendance records exist
```

```
  IF records are available THEN
    Calculate the required attendance measures
    Apply the predefined attendance conditions

    IF a specified condition is met THEN
      Identify the relevant attendance pattern
    ELSE
      Report no matching pattern
    END IF

    Display the result
  ELSE
    Display a message indicating insufficient records
  END IF
END
```

This process describes the general analysis workflow. The specific attendance calculations and conditions should match those implemented in the project's Python files.

10. Dashboard and Data Presentation Module

The Dashboard and Data Presentation Module displays relevant information according to the user's role. It provides a central place for users to access the information and features available to them.

The Administrator dashboard supports administrative activities, while the Teacher dashboard provides access to student-related academic functions. The Student and Parent dashboards present relevant progress information according to the implemented access rules and available records.

Django views retrieve information from the database and pass it to HTML templates. Bootstrap and CSS are used to organize the displayed content, while Django's template system renders the information on the page.

Classes, Functions, and Methods:

- **Dashboard Views:** Retrieve the information required for each dashboard.
- **Django ORM Methods:** Fetch the relevant database records.
- **Authentication and Permission Checks:** Restrict access to appropriate users.
- **Django Templates:** Display the information passed by the views.
- **Bootstrap and CSS:** Provide the layout and visual presentation.

Pseudocode: Display Role-Based Dashboard

```
BEGIN
  Check whether the user is authenticated

  IF the user is authenticated THEN
    Identify the user's role

    IF role is Administrator THEN
      Load administrator dashboard
    ELSE IF role is Teacher THEN
      Load teacher dashboard
    ELSE IF role is Student THEN
      Load student dashboard
    ELSE IF role is Parent THEN
      Load parent dashboard
    ELSE
      Deny access
    END IF
  ELSE
    Redirect to login page
  END IF
END
```

11. Student Progress Report Generation Module

The Student Progress Report Generation Module collects relevant student information and presents it in a structured report. The report can include academic results, attendance records, and other available student progress information.

The student report functionality retrieves the required records through Django views and the ORM. The project also uses ReportLab to generate PDF documents, allowing the relevant report information to be presented in a downloadable document.

The report generation process depends on the information stored in the database and the data selected by the user. Permission checks are important because student reports may contain personal and academic information.

Classes, Functions, and Methods:

- **Student Report View:** The student report functionality is connected through the student-report route in core/urls.py.
- **Django ORM Methods:** Retrieve the student's academic and attendance records.
- **ReportLab Library:** Generates PDF documents from the prepared report information.
- **PDF Export View:** The export functionality is connected through the export-pdf route.
- **Permission Checks:** Control access to student information and report generation.
- **Templates and Report Formatting Logic:** Organize report information for display or PDF output.

Pseudocode: Generate Student Progress Report

```
BEGIN
  Receive the student identification
  Verify user permission

  IF access is permitted THEN
    Retrieve student information
    Retrieve grade records
    Retrieve attendance records
    Retrieve other available progress information

    Prepare the report content

    IF a PDF report is requested THEN
      Generate the PDF using ReportLab
      Return the generated document
    ELSE
      Display the report on the page
    END IF
  ELSE
    Deny access
  END IF
END
```

The generated report presents the information available in the system. It does not guarantee that every report contains behavior or comment information unless those records are explicitly retrieved and included in the report implementation.

12. Database Management and ORM Operations

The Database Management Module stores and retrieves the information used throughout SPMS. SQLite is used as the database, while Django models define the structure of the stored records.

Django's Object-Relational Mapper (ORM) allows views and other application components to perform database operations using Python methods. This approach reduces the need to write raw SQL queries for common operations and helps maintain consistent access to stored information.

Database operations include creating records, retrieving existing information, updating records, and deleting records where the corresponding functionality is available. Django migrations are used to apply changes to the database structure when models are modified.

Classes, Functions, and Methods:

- **Django Models:** Define the data structure for student and academic information.
- **ORM Query Methods:** Retrieve, create, update, and delete database records.
- **Database Migrations:** Apply model changes to the database schema.
- **View Functions:** Coordinate database operations with application requests.
- **Validation Logic:** Helps ensure that submitted information meets the application's requirements.

Pseudocode: Database Record Management

```
BEGIN
  Receive a database operation request
  Verify the required permissions
  Validate the submitted information

  IF the request is valid THEN
    Perform the requested database operation

    IF the operation is successful THEN
      Return a success response
    ELSE
      Return an error response
```

```
        END IF
    ELSE
        Return a validation error
    END IF
END
```

Summary

The implementation of SPMS is divided into modules to make the system easier to understand, maintain, and extend. Django models and ORM methods manage stored information, views process user requests, and templates display the results. The authentication and role management functions control access to different features.

The grade, attendance, behavior, and comment modules maintain student-related information. Performance and attendance analysis use predefined statistical calculations and rule-based conditions to provide insights into student progress. Finally, dashboards and PDF report generation help present the available information in a structured form.

The pseudocode included in this chapter explains the main processing steps of the system. It provides a simplified representation of the application logic rather than the complete Python source code.

4.2 Testing

Testing is performed to verify whether the implemented system functions operate according to the defined requirements. The testing covers individual functions as well as the overall interaction of the system. The main areas tested include authentication, role-based access, academic records, attendance, performance analysis, risk analysis, recommendations, and report generation.

4.2.1 Unit Testing

Unit testing was conducted to check the individual functions of the Student Progress Management System (SPMS), including authentication, role-based access, grade calculation, attendance management, performance analysis, risk calculation, improvement suggestions, and PDF report generation.

Table 4.6 Unit Testing of Student Progress Management System

Test Case ID	Test Description	Input Test Data	Expected Result	Actual Result	Status
UT-01	Valid user authentication	Valid credentials of an existing user	User is authenticated and redirected to the appropriate dashboard.	Login works successfully and the user is redirected to the appropriate dashboard.	Pass
UT-02	Role-based access checking	Administrator, teacher, student, and parent roles	Each role can access its appropriate dashboard.	The role-selection page provides access to the administrator, teacher, student, and parent portals.	Pass
UT-03	Grade entry and percentage calculation	Anish Baraili, Mathematics, 97 marks out of 100	Percentage = (97/100) × 100 = 97%.	Grade record was saved successfully with 97/100 marks and 97.0%.	Pass
UT-04	Record student attendance	Anish Baraili, 6 October 2026, Present	Attendance is saved for the selected student and date.	Attendance record was saved successfully with the status Present.	Pass
UT-05	Calculate student performance	Anish Baraili's marks: 97 and 95	Average = (97 + 95)/2 = 96%. Performance	Student report displays a grade average of	Pass

			level: Excellent. Number of assessments: 2.	96.0% and two Mathematics assessment records.	
UT-06	Calculate student risk score	Average marks = 96%; attendance = 83.33%	Academic risk = 0; attendance risk = 20; total risk score = min(0 + 20, 100) = 20.	Student report displays a risk score of 20%.	Pass
UT-07	Classify student risk level	Risk score = 20	Risk level is Low Risk because the score is between 20 and less than 40.	Student report displays Low Risk.	Pass
UT-08	Calculate attendance rate	5 present days and 1 absent day	Attendance rate = (5/6) × 100 = 83.33%.	Student report displays an attendance rate of 83.3%.	Pass
UT-09	Classify attendance risk	Attendance rate = 83.33%	Attendance risk is High because the attendance rate is below 85%. The separate below-80% condition is not triggered.	Student report displays the insight “Poor attendance affecting performance.”	Pass
UT-10	Generate improvement suggestions	Average marks = 96%; attendance = 83.33%; risk score = 20	The system provides a positive progress suggestion and	Student report displays “Maintain current performance”	Pass

			identifies attendance as an area for improvement.	and “Poor attendance affecting performance.”	
UT-11	Generate comprehensive performance analysis	Average marks = 96%; attendance = 83.33%; risk score = 20	Overall status is Minor Concerns, with a Low Risk classification.	Student report displays Minor Concerns, a risk score of 20%, a grade average of 96.0%, attendance of 83.3%, and Low Risk.	Pass
UT-12	Generate student PDF report	Anish Baraili's student details, Mathematics grades, and risk analysis	A PDF report is generated containing the available student and analysis information.	PDF report was generated successfully and displayed the student name, roll number, class, recent grades, risk level, and overall status.	Pass

Pass is provisional for UT-03 and UT-04 because the provided admin records show that the corresponding records exist, but they do not independently prove that every part of the test was executed and passed.

4.2.2 System Testing

System testing verifies the overall behavior of the application and the interaction between its major modules. The testing includes authentication, role-based access, academic record management, attendance management, performance analysis, risk analysis, dashboards, and PDF report generation.

Table 4.7 System Testing of Student Progress Management System

Test ID	Test Case	Input Test Data	Expected Result	Actual Result	Status
ST-01	Login with valid credentials	Username: admin; Password: admin123	Administrator logs in successfully.	Administrator login worked successfully.	Pass
ST-02	Role-based access	Admin: admin / admin123; Teacher: teacher1 / teacher123; Student: student1 / student123; Parent: parent1 / parent123	Each user accesses the appropriate portal according to their role.	You confirmed that all four roles were tested successfully.	Pass
ST-03	Teacher adds a grade	Teacher account: teacher1; Student: Anish Baraili; Roll No.: 208302; Class: 10-A; Subject: Mathematics; Marks: 97/100; Assessment:	The grade is saved and displayed in the student's records.	Teacher portal grade entry worked successfully. The Mathematics Final grade was recorded as 97/100.	Pass

		Final; Date: 2026-10-06			
ST-04	Teacher records attendance	Teacher account: teacher1; Student: Anish Baraili; Roll No.: 208302; Date: 2026-10-06; Status: Present	Attendance is saved for the selected student and date.	Teacher portal attendance entry worked successfully. Anish's attendance record for October 6 was Present.	Pass
ST-05	View student performance report	Student: Anish Baraili; Mathematics Final: 97/100; Mathematics Quiz: 95/100; Number of assessments: 2	The system displays the grades and calculates the average correctly.	The report displayed 97/100 and 95/100, with a grade average of 96.0%.	Pass
ST-06	View student risk analysis	Student: Anish Baraili; Grade average: 96.0%; Attendance: 83.3%	The system displays the risk score and risk classification.	The report displayed a risk score of 20% and Low Risk.	Pass

ST-07	View attendance analysis	Student: Anish Baraili; Present days: 5; Absent days: 1; Total days: 6	The system calculates the attendance rate and displays attendance analysis.	The report displayed 83.3% attendance and the insight “Poor attendance affecting performance.”	Pass
ST-08	View improvement suggestions	Student: Anish Baraili; Grade average: 96.0%; Attendance: 83.3%; Risk score: 20%	The system displays a recommendation based on the student’s analysis.	The report displayed “Maintain current performance.”	Pass
ST-09	Student accesses personal analysis	Student login: student1 / student123; access the student’s own analysis page	The student can view their own permitted analysis.	You confirmed that the student successfully accessed their own analysis.	Pass
ST-10	Parent views associated child’s information	Parent login: parent1 / parent123; test the parent account with	The parent can view the associated child’s information.	You confirmed that the parent dashboard displayed the associated	Pass

		its associated child		child's information.	
ST-11	Generate student PDF report	Student: Anish Baraili; Roll No.: 208302; Class: 10-A; Grades: 97/100 and 95/100	The system generates a PDF containing the student's report information.	The PDF was generated successfully and displayed recent grades, Low Risk, and Minor Concerns.	Pass
ST-12	Logout	Logged-in administrator account: admin; select Logout	The session ends and the user returns to the portal selection page.	After logout, the system redirected to the four-role selection page.	Pass

The system testing verifies the interaction among authentication, role-based access, database records, analysis functions, dashboards, and report generation.

4.3 Result Analysis

The results of the Student Progress Management System (SPMS) were analyzed using the sample student records entered into the system. The dataset included 12 students, 18 grade records, and 72 attendance records collected across six dates. These records were used to check the system's grade calculations, attendance analysis, risk classification, recommendations, and report generation.

4.3.1. Academic Performance Analysis

The system calculated student performance using the available grade records. Among the entered records, marks ranged from 20 to 99 out of 100. The system calculated the average marks for each student based on their available assessments and assigned a performance level using the predefined conditions.

For example, Anish Baraili had two Mathematics assessments: 97 out of 100 in the Final assessment and 95 out of 100 in the Quiz. The calculated average was:

Average = (97 + 95) / 2 = 96%

The system classified this result as Excellent, based on the defined performance criteria. This result shows that the system can calculate average marks and present the student's academic performance using the available grade data.

4.3.2. Attendance Analysis

The attendance dataset contained 72 records for 12 students across six dates. Out of these records, 35 were marked Present and 37 were marked Absent. The overall attendance rate for the entered dataset was calculated as follows:

Attendance Rate = (35 / 72) × 100 = 48.61%

For Anish Baraili, five out of six attendance records were marked Present, while one was marked Absent. His attendance rate was therefore 83.33%. The system displayed an attendance rate of 83.3% and the insight “Poor attendance affecting performance.”

These results demonstrate that the system calculates attendance percentages from the stored records and displays attendance-related information for individual students.

4.3.3. Risk Analysis and Recommendations

The system used predefined rules to calculate risk scores from recent academic and attendance records. For Anish Baraili, the average grade was 96%, which contributed zero academic risk points. His attendance rate of 83.33% contributed 20 attendance risk points. Therefore:

Risk Score = Academic Risk + Attendance Risk

Risk Score = 0 + 20 = 20

The system displayed a risk score of 20% and classified the student as Low Risk. The report also displayed the overall status “Minor Concerns” and the recommendation “Maintain current performance.”

This example shows how SPMS combines grade and attendance information to produce a risk score, assign a risk level, and display a recommendation based on its predefined rules.

4.3.4. Student Report Generation

The system successfully generated a PDF report for Anish Baraili. The report included the student's roll number, class, recent Mathematics grades, risk level, and overall status. It displayed the two assessment results of 97/100 and 95/100, along with the classifications “Low Risk” and “Minor Concerns.”

The successful generation of this report demonstrates that the system can retrieve stored student information and present the available academic and analysis results in a structured document.

4.3.5. Overall Result

Based on the entered sample data and the tested system functions, SPMS was able to store and display student grade and attendance records, calculate grade averages and attendance rates, classify student risk levels, display recommendations, and generate PDF reports. The system also provided separate access for administrators, teachers, students, and parents according to their roles.

The analysis is based on the sample records entered during development and testing. Since the attendance dataset covers only six dates and the number of academic assessments is limited, these results should not be treated as a complete evaluation of student performance over a full academic year.

The system uses statistical calculations and predefined rules rather than a trained machine learning model. Therefore, no machine learning accuracy percentage or measured improvement in student performance is claimed. The results depend on the accuracy and availability of the data entered into the system.

Table 4.8 Summary of Sample Data and Analysis Results of Student Progress Management System

Measurement	**Actual Result**
Students in the attendance dataset	12
Grade records entered	18
Attendance records entered	72
Present attendance records	35
Absent attendance records	37
Overall attendance rate	48.61%
Anish Baraili's Mathematics average	96.0%
Anish Baraili's attendance rate	83.33%
Anish Baraili's risk score	20
Anish Baraili's risk classification	Low Risk
Anish Baraili's overall status	Minor Concerns
PDF report generation	Successful

Chapter 5: Conclusion and Future Recommendations

5.1 Lesson Learnt / Outcome

The development of the Student Progress Management System (SPMS) provided practical experience in analyzing, designing, developing, and testing a web-based application. The project provided an understanding of how a student progress management system can be developed by integrating user management, database operations, academic records, attendance records, performance analysis, and report generation into a single application.

The project provided practical experience in using Python and Django for web application development. It also helped in understanding database design and relationships using SQLite, including the management of students, teachers, parents, classes, subjects, grades, and attendance, along with the storage of behavior and comment information. The implementation of role-based access provided experience in managing different system functions for administrators, teachers, students, and parents.

The project also provided experience in implementing predefined rules for analyzing grade and attendance information. The system uses statistical calculations and predefined conditions to calculate a risk score, assign a risk level, classify performance, analyze attendance, and provide improvement suggestions based on available records. This helped in understanding how existing student data can be processed to provide structured information for monitoring academic progress.

Another outcome of the project was practical experience in generating PDF-based student progress reports using ReportLab. The development process also provided experience in testing system functions, identifying errors, and improving the system based on the requirements of the project.

Overall, the project provided practical knowledge of the complete software development process, from requirement analysis and system design to implementation and testing. It also demonstrated how a web-based application can be used to organize student information and support the monitoring of academic progress.

5.2 Conclusion

The Student Progress Management System (SPMS) was developed as a web-based system for managing and monitoring students' academic and related information in a centralized environment. The system provides separate access for administrators, teachers, students, and parents according to their roles. It manages student information, classes, subjects,

grades, attendance, and behavior records, while the database also includes comment information.

The system provides teachers with functions to enter grades and record attendance, while students and parents can view relevant academic information and performance analysis. Role-based dashboards provide summarized information according to the user's role. The system also generates PDF reports containing student information, academic records, percentages, and performance analysis.

A rule-based and statistical analysis approach was implemented to analyze available grade and attendance information. The system calculates a risk score using predefined conditions and assigns a risk level such as High Risk, Medium Risk, Low Risk, or On Track. It also provides performance classifications, attendance analysis, and improvement suggestions based on the available student data. This provides a structured approach for identifying areas that may require attention.

The development of the system demonstrates the use of Python and Django for web application development, SQLite for data storage, and ReportLab for PDF report generation. The completed system provides the main functions required for centralized student progress management and presents academic information in an organized manner. The analysis is based on predefined rules and available records rather than a trained machine learning model; therefore, the analysis results depend on the quality and amount of data entered into the system.

5.3 Future Recommendations

The Student Progress Management System can be further improved by adding additional features and technologies. The following recommendations can be considered for future development:

- **Machine Learning Integration:** A trained machine learning model can be integrated to provide more advanced student performance prediction using a larger historical dataset.
- **Improved Trend Analysis:** Future versions can include analysis of improving and declining performance trends across multiple assessments.
- **Assignment Management:** Assignment creation, submission, grading, and tracking can be added to provide more complete academic management.
- **Notification System:** Email or in-system notifications can be introduced to inform students, parents, and teachers about important academic or attendance updates.

- **Enhanced Reporting:** More detailed reports and graphical representations of academic and attendance information can be added.
- **Database Scalability:** A more scalable database system can be considered when the number of users and records increases.
- **Security Improvements:** Additional security measures such as stronger input validation, improved session protection, and appropriate security configuration can be implemented.
- **Mobile Application:** A mobile application or mobile-friendly interface can be developed to provide easier access to the system on smartphones.
- **Historical Data Analysis:** Long-term student records can be used to provide more detailed analysis of academic progress and support data-based decision-making.

References

[1] J. Martins, F. Branco, R. Gonçalves, M. Au-Yong-Oliveira, T. Oliveira, M. Naranjo-Zolotov and F. Cruz-Jesus, "Assessing the success behind the use of education management information systems in higher education," *Telematics and Informatics,* vol. 38, p. 182–193, 2019.

[2] M. A. Kraft and T. Rogers, "The underutilized potential of teacher-to-parent communication: Evidence from a field experiment," *Economics of Education Review,* vol. 47, p. 49–63, 2015.

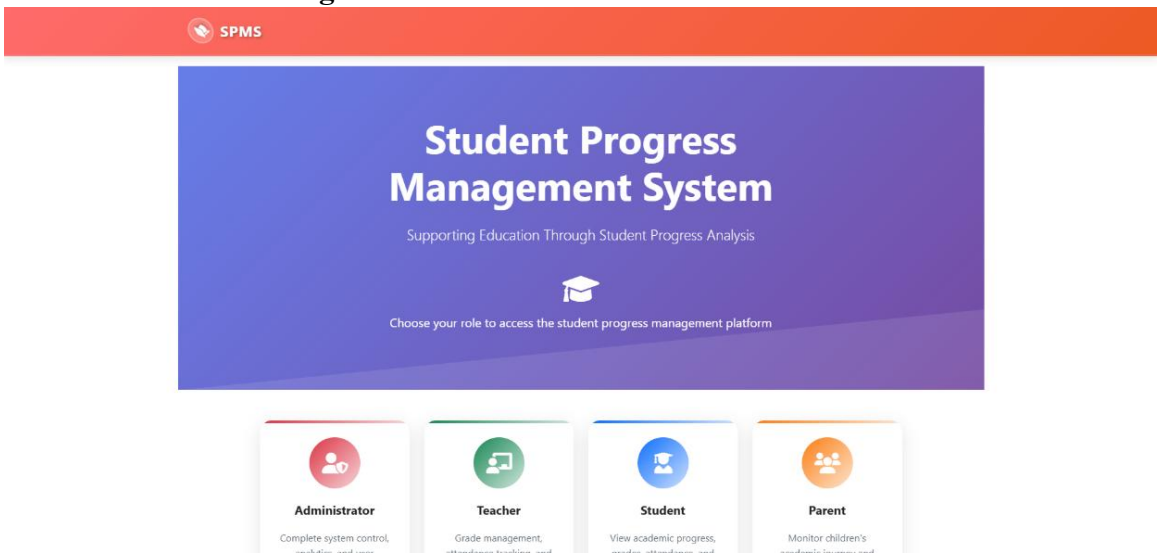
[3] A. Namoun and A. Alshanqiti, "Predicting student performance using data mining and learning analytics techniques: A systematic literature review," *Applied Sciences,* vol. 11, no. 1, p. 237, 2021.

[4] M. H. Bin Roslan and C. J. Chen, "Educational data mining for student performance prediction: A systematic literature review (2015–2021)," *International Journal of Emerging Technologies in Learning (iJET),* vol. 17, no. 5, p. 147–179, 2022.

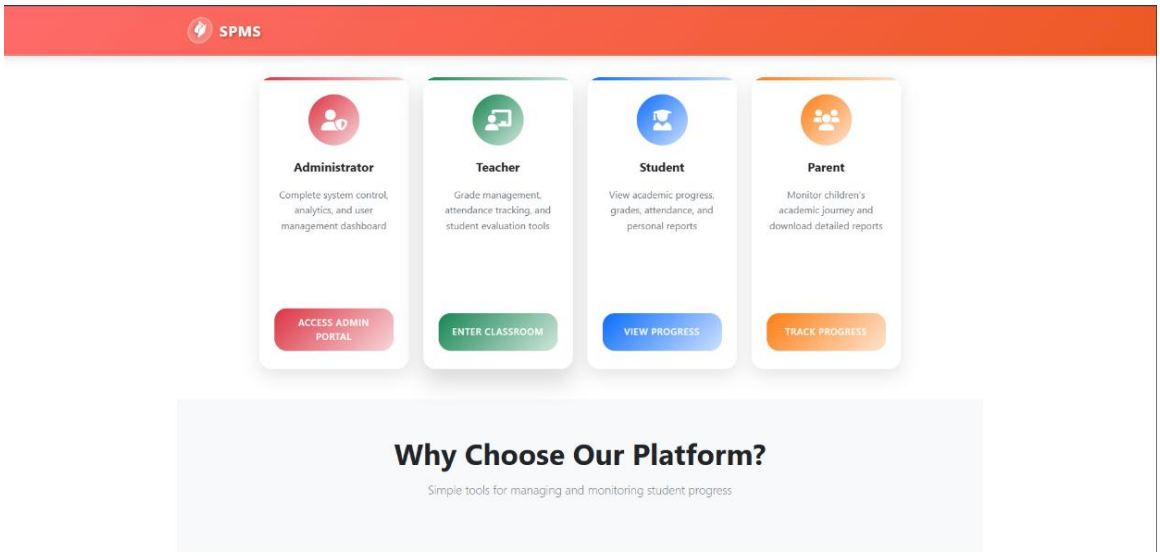
[5] K. Mohammed, A. S. Tolba and M. Elmogy, "Multimodal student attendance management system (MSAMS)," *Ain Shams Engineering Journal,* vol. 9, no. 4, p. 2917–2929, 2018.

Appendices

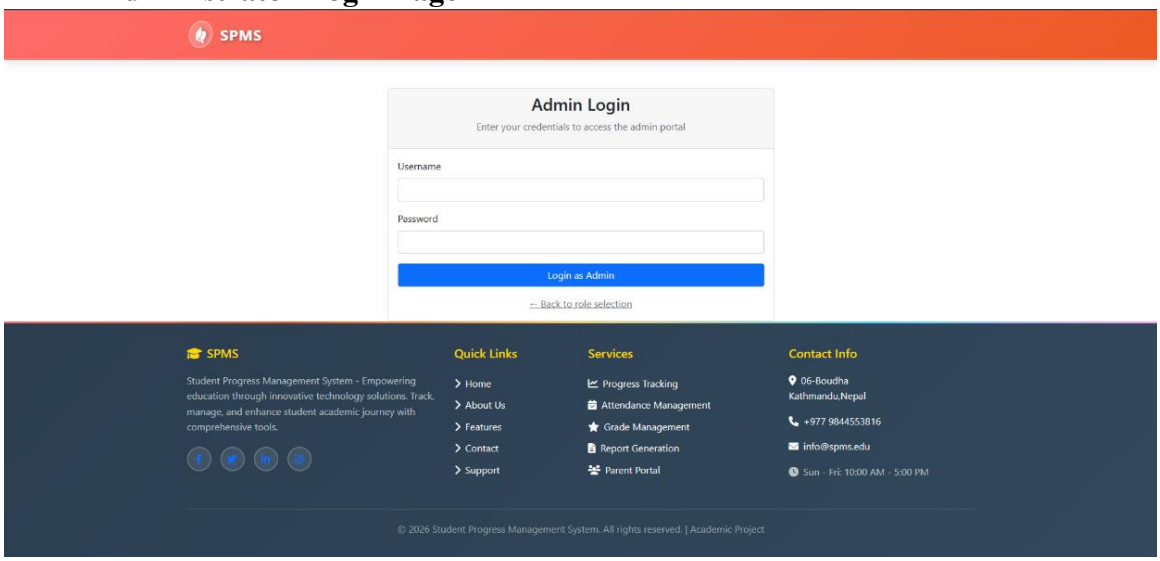
- **SPMS Home Page**



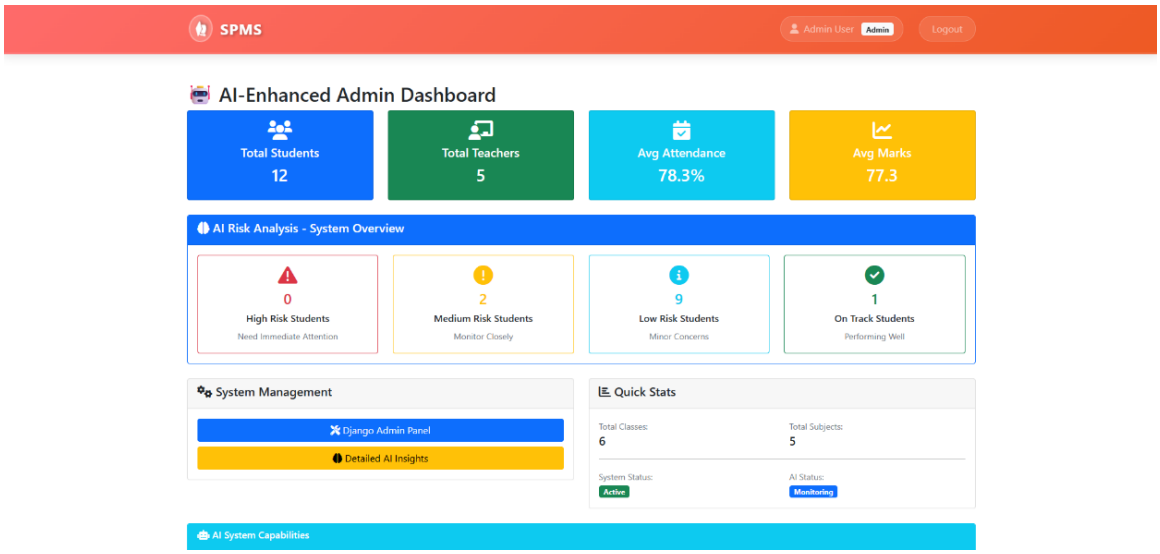
- **User Role Selection and Platform Features**



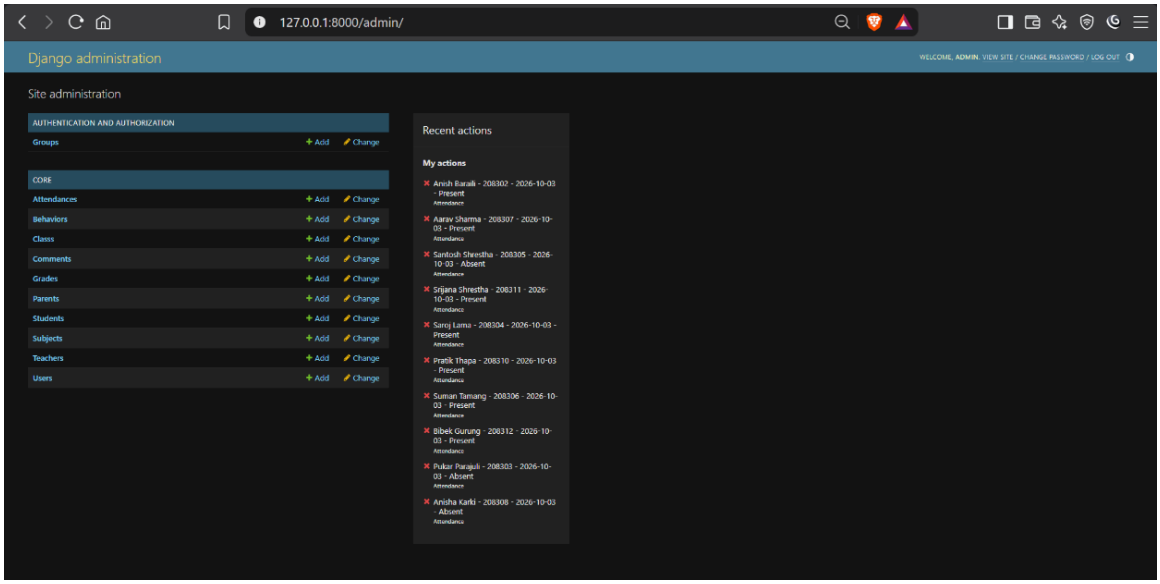
- **Administrator Login Page**



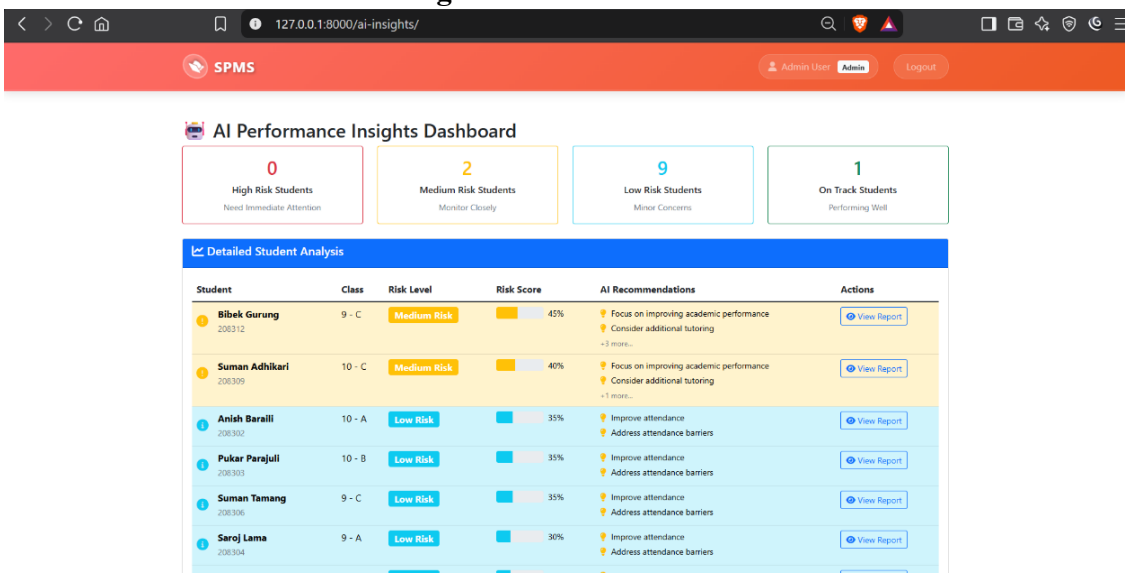
- **Administrator Dashboard**



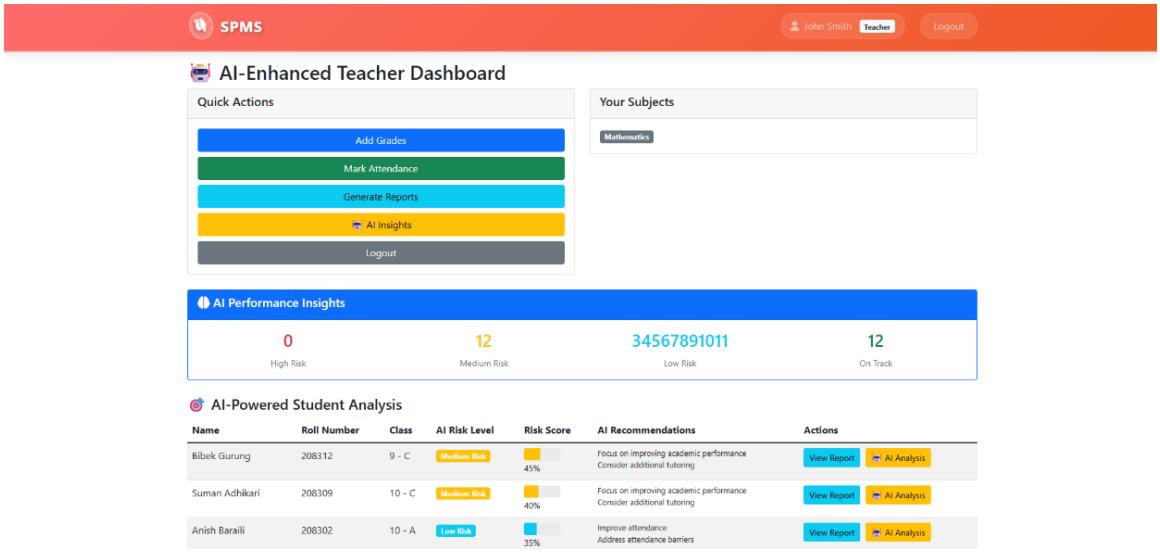
- **Django Administration Panel**



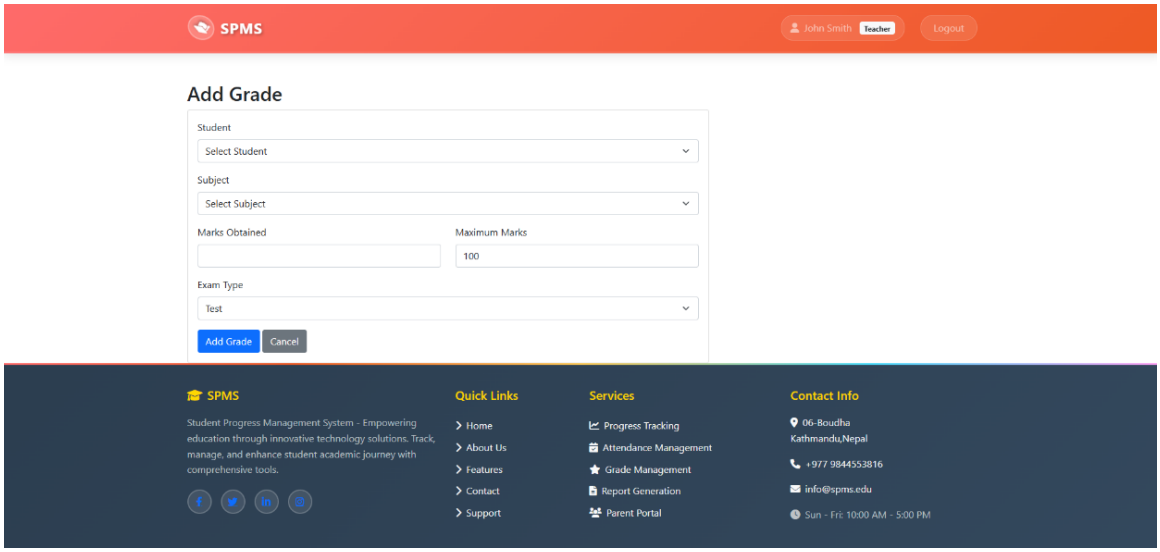
- **Student Performance Insights Dashboard**



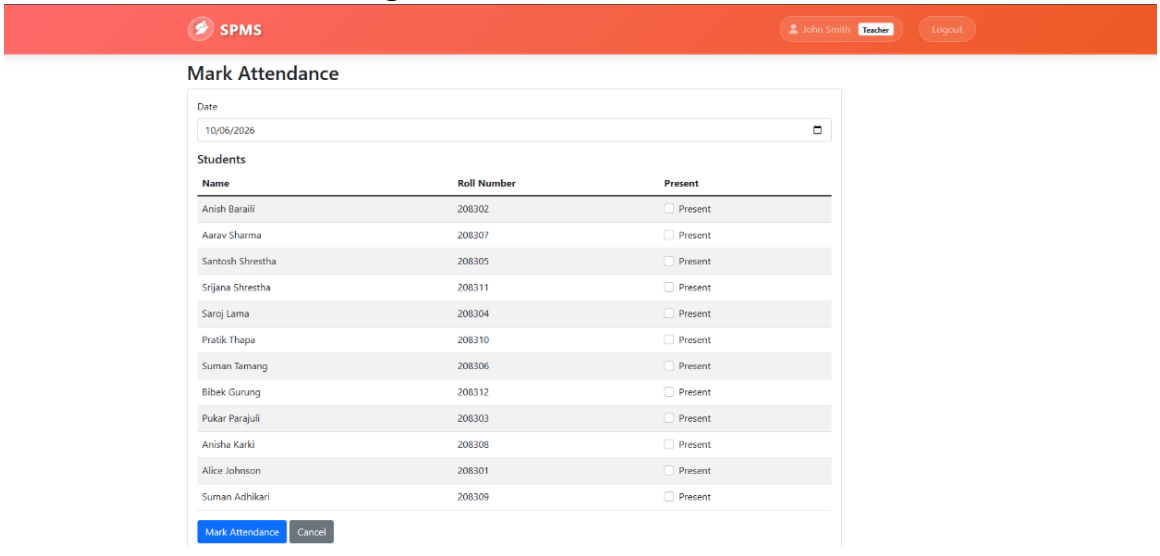
- **Teacher Dashboard**



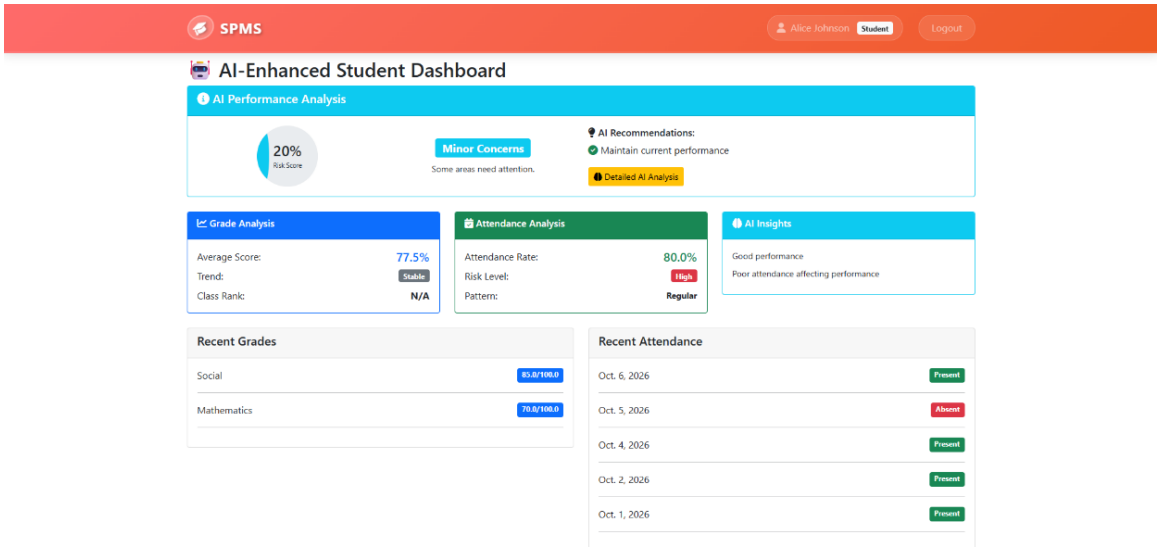
- **Add Grade Page**



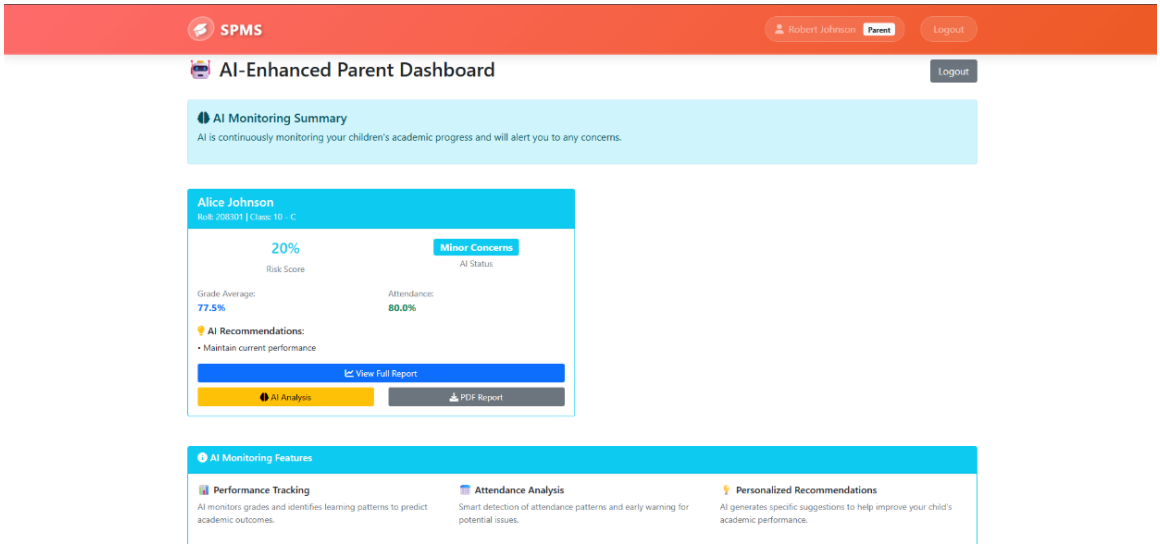
- **Mark Attendance Page**



- **Student Dashboard**



- **Parent Dashboard**



Student Progress Report PDF

